

Memory Management



Politecnico
di Torino

Department of Control and
Computer Engineering



System Programming - Sarah Azimi

CAD & Reliability Group
DAUIN- Politecnico di Torino

Objectives

- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques

Memory Management

- Background
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Swapping
- Example: The Intel 32 and 64-bit Architectures
- Example: ARMv8 Architecture

Today:

- Why do we need Memory Management?
- Protection (one process does not access the memory of another process)
- Logical Memory vs. Physical Memory
- MMU
- Variable Partition – First Fit, Best Fit, Worst Fit
- Fragmentation

Background

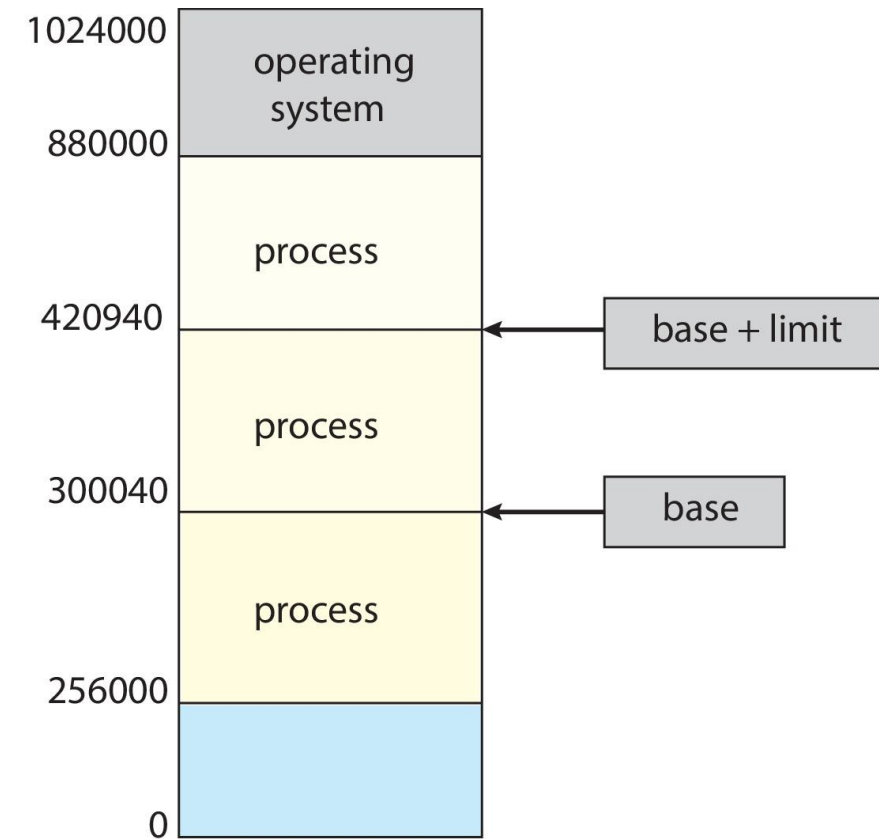
- Program must be brought (from disk) into memory and placed within a process for it to be run.
- Main memory and registers are only storage CPU can access directly.
- An instructions in execution and any data being used by them, must be in one of the direct access memory.
 - Register access is done in one CPU clock (or less)
 - Main memory can take many cycles, so the processor does not have the data to complete the instruction, causing a **stall**
 - **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

Protection

- Need to ensure that a process can access only those addresses in its address space.
- Protecting the processes from each other which is fundamental for having multiple processes for concurrent executions.
- We can provide this protection by using a pair of **base** and **limit registers** define the logical address space of a process

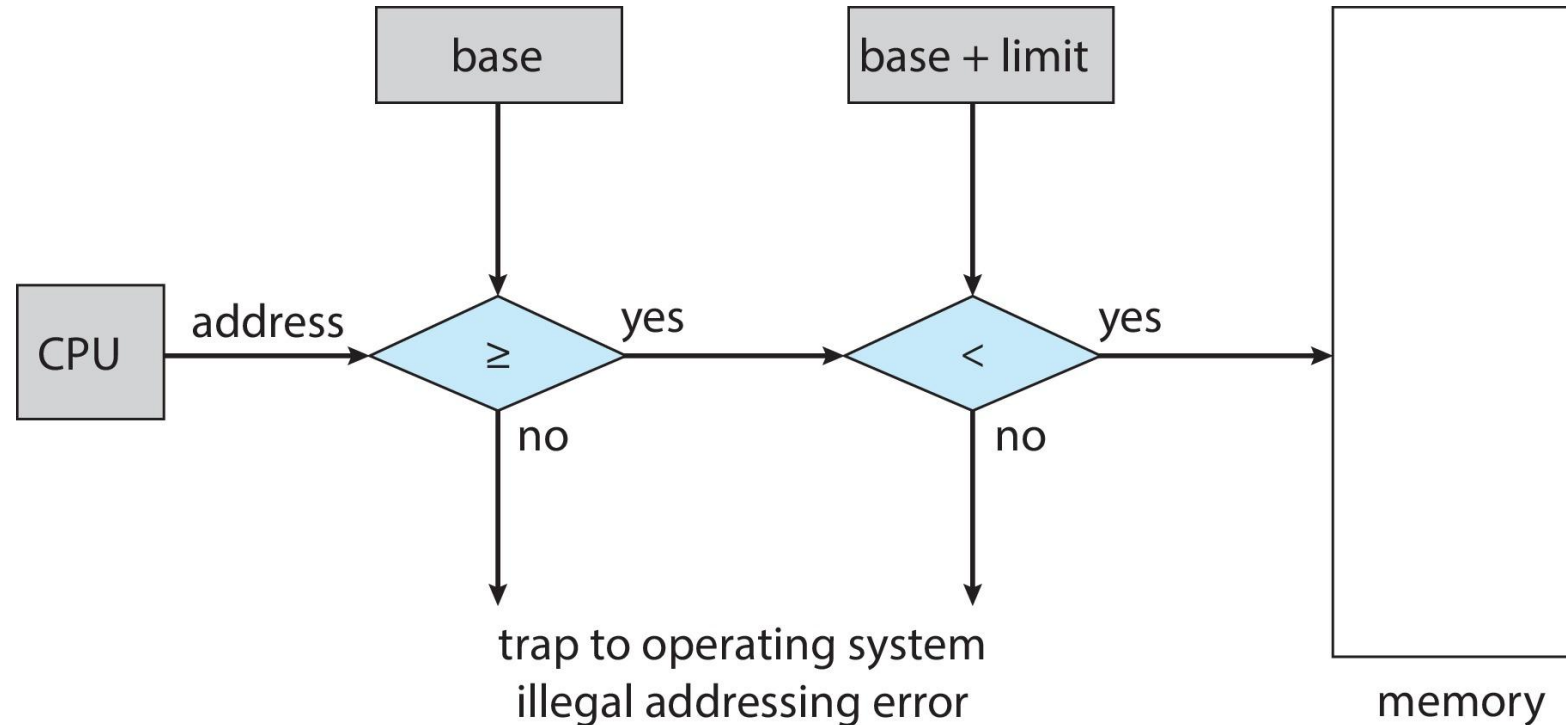
Protection

- To separate memory spaces, we need to determine the range of legal addresses that the process may access and to ensure that the process can access only these legal addresses.
- We can provide this protection by using a pair of **base** and **limit registers** to define the address space of a process.
 - The **base register** specifies the smallest legal physical memory addresses.
 - The **limit register** specifies the size of the range.



Hardware Address Protection

- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



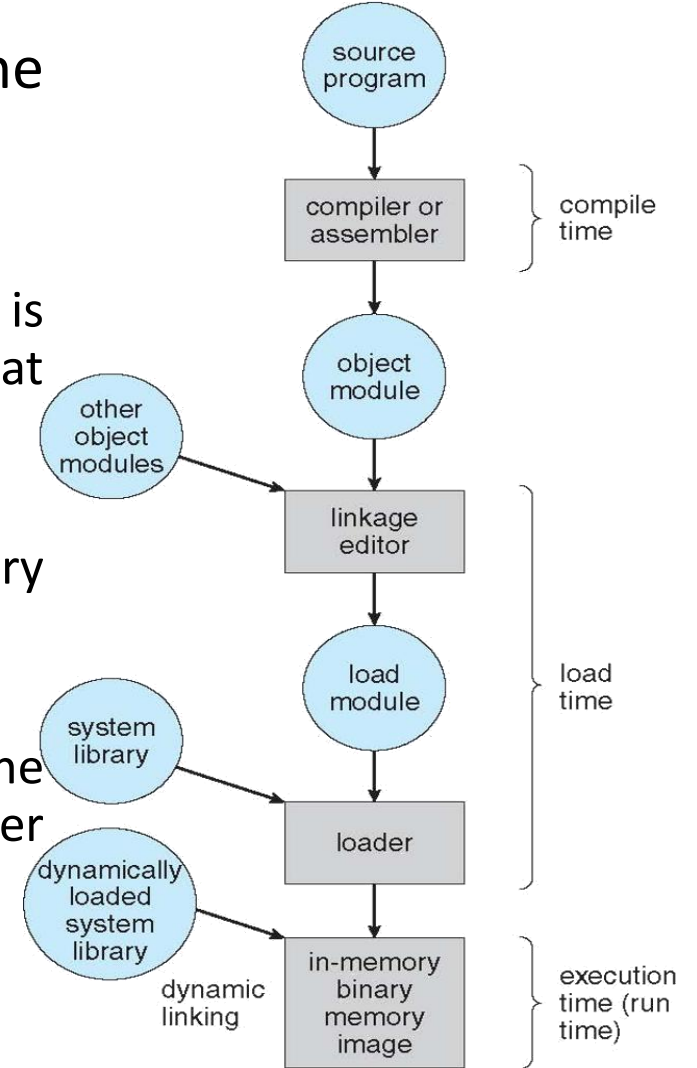
- The base and limit registers can be loaded only by the operating systems using special privileged instructions.

Address Binding

- Usually, a program is on disk as a binary executable file.
- To run, it should be brought into memory to be eligible for execution on the CPU.
- Most systems allow a user process to stay in any part of the physical memory, for example at 0000.
- Addresses in the source program are generally symbolic.
- A compiler typically **binds** these symbolic addresses to relocatable addresses.
- **Address binding in an operating system refers to the process of assigning a memory address to a program or a process at the time of execution.**

Multistep Processing of a User Program

- Compile time, load time, and run-time are important stages in the life cycle of a program:
 - **Compile time:** This is the phase in which the source code of a program is converted into machine code. The machine code is an executable file that can be run on the target system.
 - **Load time:** After the executable file is created, it is loaded into memory when it is executed.
 - **Run time:** This is the phase in which the program is executed. During the run time, the program interacts with the operating system and other processes to perform its tasks.



Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - **Load time:** if memory location is not known at compile time, then the compiler must generate **relocatable code** time. In this case, the final binding is delayed until load time.
 - **Execution time:** if the process can be moved during its execution from one memory segment to another, then binding delayed until run time.
 - Need hardware support for address maps (e.g., base and limit registers)

Dynamic Loading

- The entire program needs to be in memory to execute.
- Routine is not loaded until it is called.
- Loading of libraries or modules into memory at runtime – **Dynamic Loading**
- Enhances efficiency by loading only necessary components when needed.
- Reduces memory footprint and potentially speeds up program startup.

Dynamic Linking

- **Static Linking** – system libraries and program code combine into the loaded into the binary program image.
- **Dynamic Linked Libraries (DLL)** are system libraries that are linked to user programs when the programs are run.
 - Collection of executable code and data
 - Shared by multiple programs simultaneously – **Shared Libraries**
- Enable sharing of resources and functionality
- Reduce memory usage and enhance modularity

Logical vs. Physical Address Space

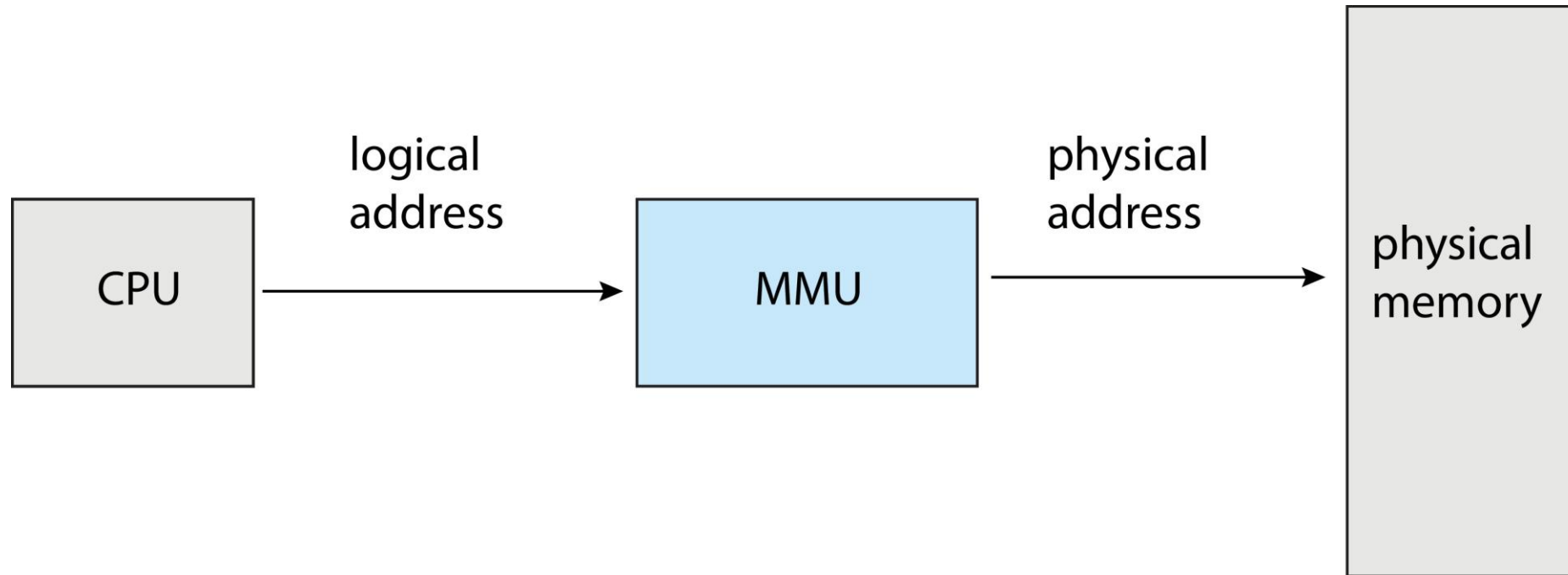
- An address generated by a CPU is commonly referred to as a **logical address**, also referred to as a **virtual address**.
- An address seen by the memory unit is referred to as a **physical address**.
- Binding addresses at the compile or load time generated identical logical and physical addresses.
- The execution time binding results in differing logical and physical addresses.
 - **Logical address space** is the set of all logical addresses generated by a program
 - **Physical address space** is the set of all physical addresses generated by a program

Logical vs. Physical Address Space

- An address generated by a CPU is commonly referred to as a **logical address**, also referred to as a **virtual address**.
- An address seen by the memory unit is referred to as a **physical address**.
- Binding addresses at the compile or load time generated identical logical and physical addresses.
- The execution time binding results in differing logical and physical addresses.
 - **Logical address space** is the set of all logical addresses generated by a program
 - **Physical address space** is the set of all physical addresses generated by a program
- The run-time mapping from the virtual to physical addresses is done by a hardware device called **Memory Management Unit (MMU)**

Memory-Management Unit (MMU)

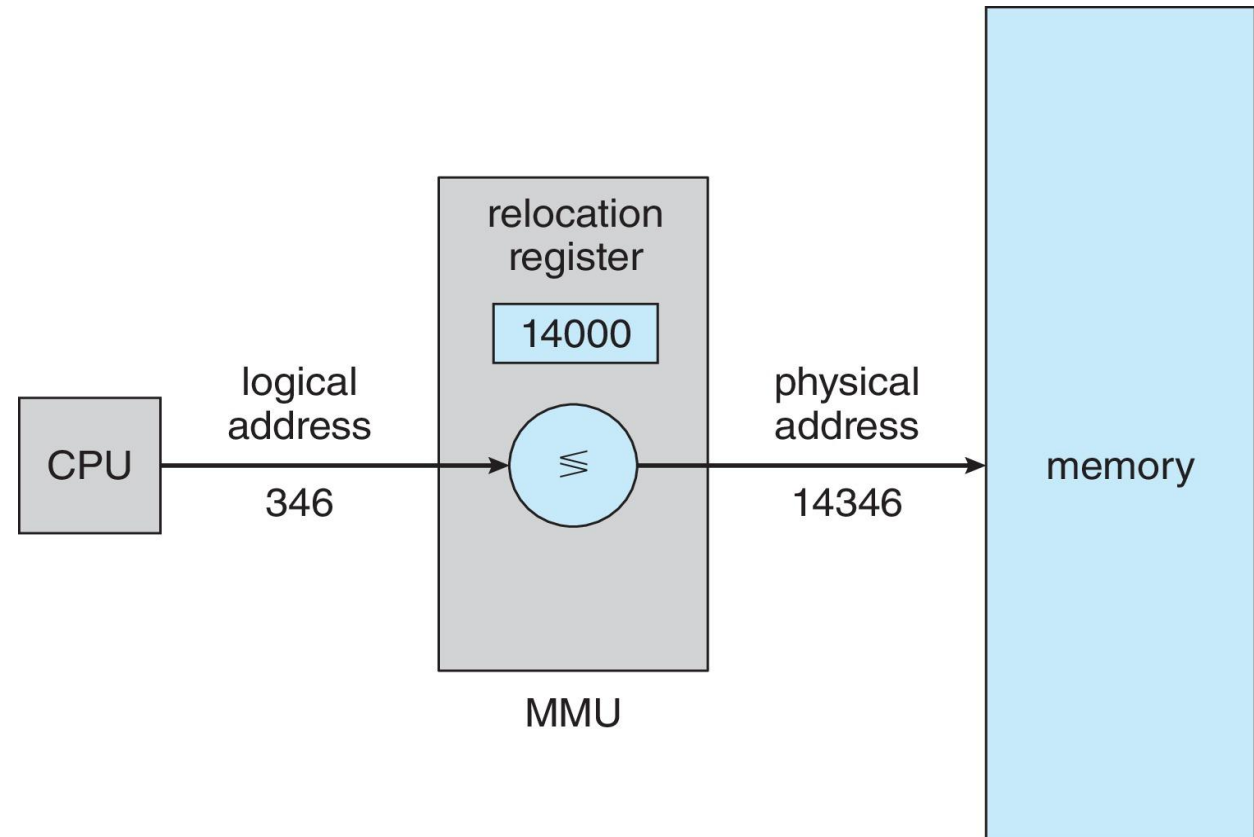
- Hardware device that at run time maps virtual to physical address



- Many methods possible, covered in the rest of this chapter

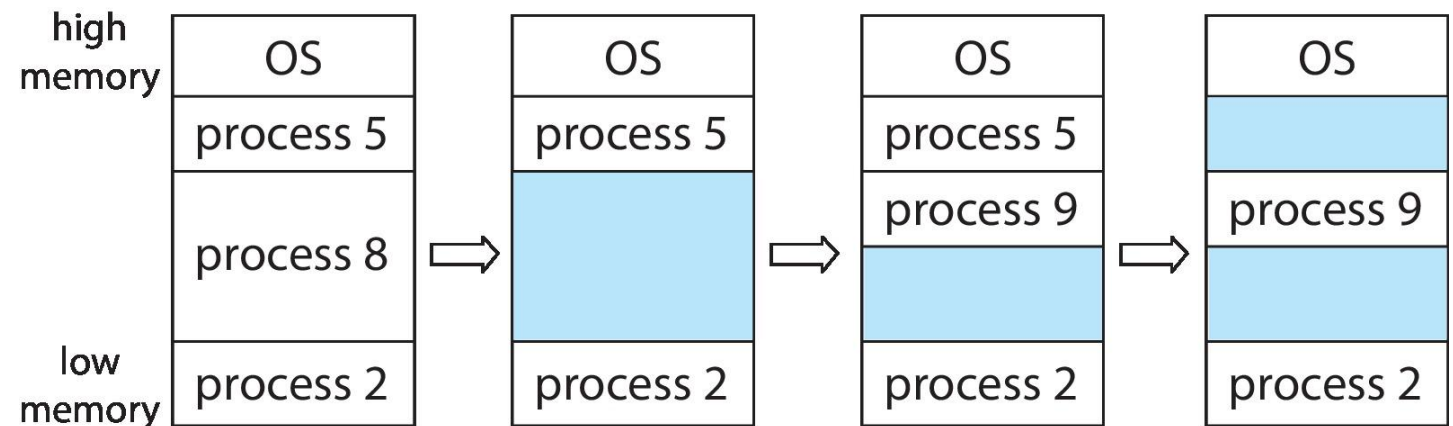
Memory-Management Unit

- Consider a simple scheme which is a generalization of the base-register scheme.
 - The base register now called the **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory



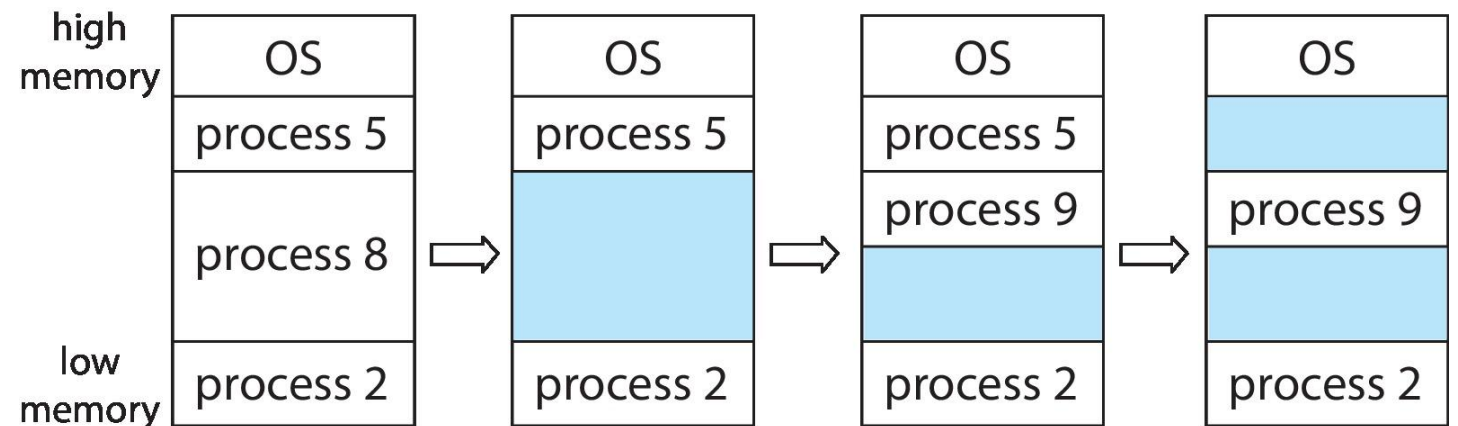
Contiguous Allocation

- Main memory must support both OS and various user processes.
- Main memory usually divides into two **partitions**:
 - Placing the OS in either low memory addresses or high memory addresses.
 - Residing the user processes in memory at the same time.



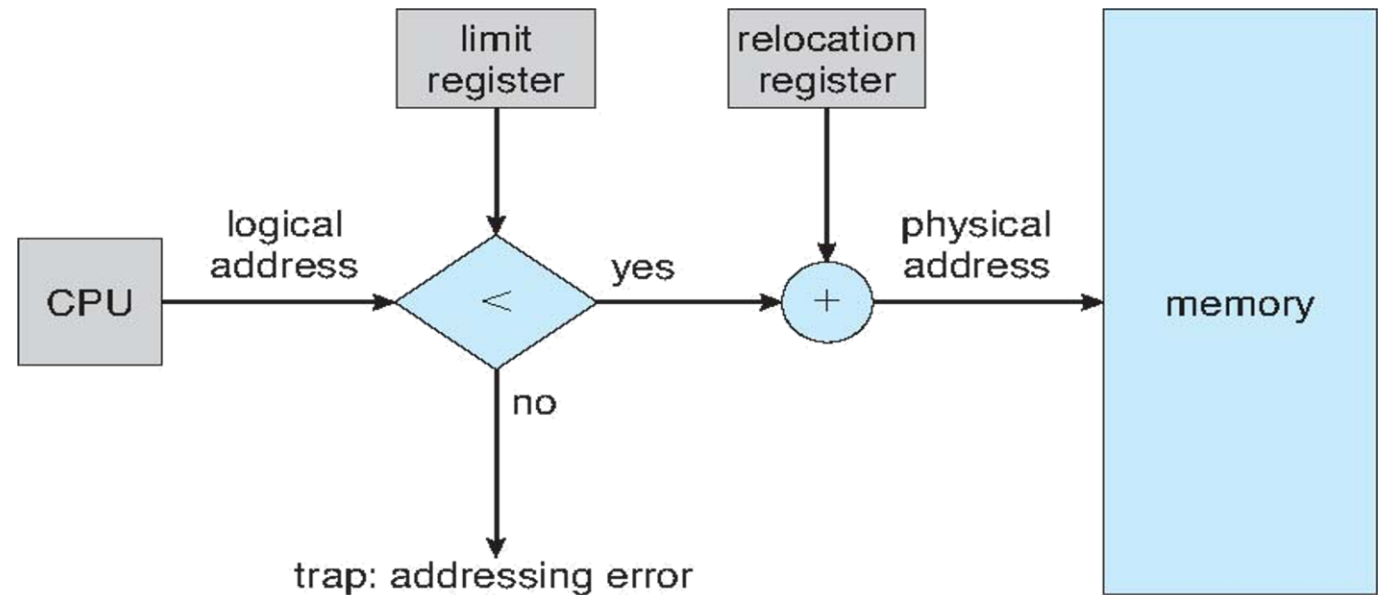
Contiguous Allocation

- How to allocate the available memory to the processes?
- In **contiguous memory allocation**, each process is contained in a single section of memory that is contiguous to the section containing the next process.



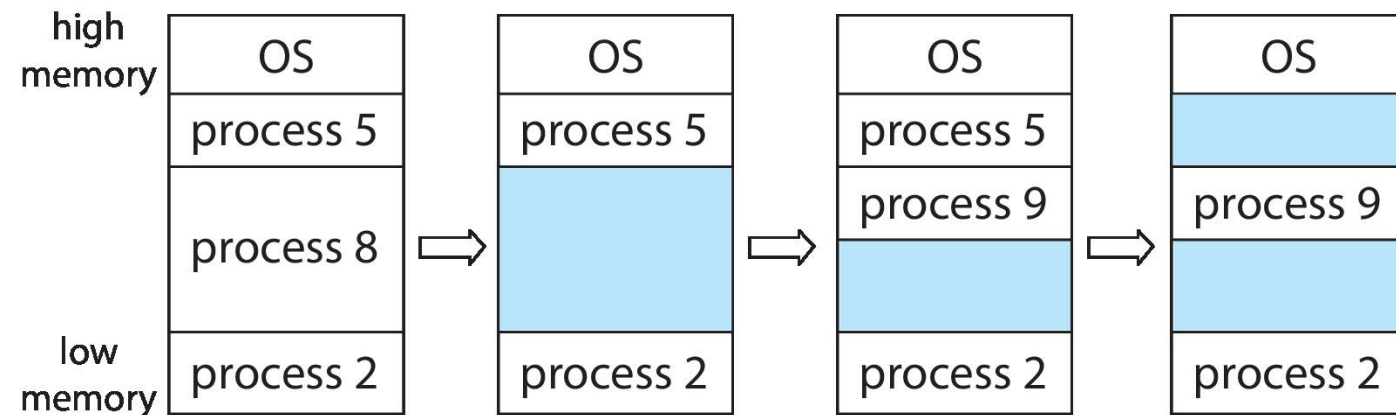
Contiguous Allocation

- **Relocation registers** used to protect user processes from each other, and from changing operating-system code and data
 - Base register contains the value of the smallest physical address
 - Limit register contains a range of logical addresses – each logical address must be less than the limit register
 - MMU maps logical addresses *dynamically*



Variable Partition

- One of the simplest methods for allocating memory is to assign processes to variably sized partitions where each partition may contain exactly one process. In **Variable partition** scheme:
 - Initially, the memory is available for user processes, considered one large block.
 - Eventually, when the process terminates, it releases its memory, creating a **hole**.
 - When a new process arrives, it is allocated memory from a hole large enough to accommodate it
- Operating system maintains information about allocated partitions and free partitions (hole)



Dynamic Storage-Allocation Problem

- When a process arrived and needs memory, the system searches for a hole that is large enough for this process.
- How to satisfy a request of size n from a list of free holes – **dynamic storage allocation problem**
 - **First-fit**: Allocate the *first* hole that is big enough
 - **Best-fit**: Allocate the *smallest* hole that is big enough; must search entire list, unless the list is ordered by size
 - Produces the smallest leftover hole
 - **Worst-fit**: Allocate the *largest* hole; must also search entire list
 - Produces the largest leftover hole which might be more useful than the smaller leftover hole from best-fit
- First-fit and best-fit better than worst-fit in terms of speed and storage utilization

Exercise:

- Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358KB, 200KB, and 375KB (in order)?

Exercise:

- Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358KB, 200KB, and 375KB (in order)?
- Answer:
 - First Fit: 115KB >> 300 KB partition, 500 KB >> 600 KB partition, 358 KB >> 750 KB partition, 200 KB >> 350 KB partition, 375 KB >> 392 KB partition.

Exercise:

- Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358KB, 200KB, and 375KB (in order)?

Exercise:

- Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358KB, 200KB, and 375KB (in order)?
- Answer:
 - Best Fit: 115KB >> 125 partition, 500 KB >> 600 KB partition, 358 KB >> 750 KB partition, 200 KB >> 200 KB partition, 375 KB >> 392 KB partition.

Exercise:

- Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358KB, 200KB, and 375KB (in order)?

Exercise:

- Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358KB, 200KB, and 375KB (in order)?
- Answer:
 - Worst Fit: 115KB >> 750 KB partition, 500 KB >> 635 KB partition, 358 KB >> 600 KB partition, 200 KB >> 350 KB partition, 375 KB >> must wait.

Fragmentation

- As processes are loaded and removed from memory, the free memory space is broken into little pieces.
- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous.
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.
- First fit analysis reveals that given N blocks allocated, $0.5 N$ blocks lost to fragmentation
 - $1/3$ may be unusable -> **50-percent rule**

Fragmentation

- Reduce external fragmentation by **compaction**
 - Shuffle memory contents to place all free memory together in one large block
 - Compaction is possible *only* if relocation is dynamic, and is done at execution time
 - I/O problem
 - Latch job in memory while it is involved in I/O
 - Do I/O only into OS buffers

Summary

- **Why do we need Memory Management?**

- Program must be brought (from disk) into memory and placed within a process for it to be run

- **Protection (one process does not access the memory of another process).**

- Need to ensure that a process can access only those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** to define the logical address space of a process

- **Logical Memory vs. Physical Memory**

- An address generated by CPU is commonly referred to as a **logical address**
- An address seen by the memory unit is referred to as **physical address**

Summary

- **MMU**

- Hardware device that at run time maps virtual to physical address

- **Contiguous memory allocation**

- each process is contained in a single section of memory that is contiguous to the section containing the next process.

- **Variable Partition – First Fit, Best Fit, Worst Fit**

- First-fit: Allocate the *first* hole that is big enough
- Best-fit: Allocate the *smallest* hole that is big enough
- Worst-fit: Allocate the *largest* hole; must also search entire list

Summary

- Fragmentation
 - As processes are loaded and removed from memory, the free memory space is broken into little pieces.
 - **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous.
 - **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.

To do:

- Paging
- Page Table
- Implementation of Page Table
- Translation look-aside buffer (TLB)
- Effective Access time
- Shared Memory

Paging

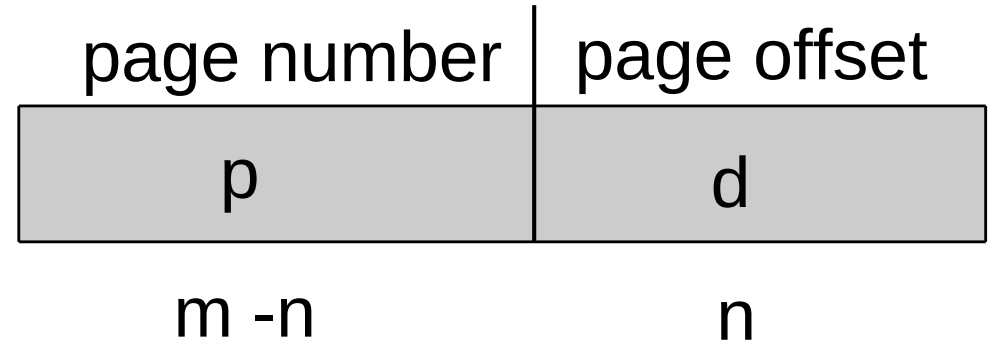
- Another possible solution to the external fragmentation problem is to permit the logical address space of processes to be **non-contiguous**.
- This is possible by adopting a strategy called **paging**.
 - Avoids external fragmentation
 - Needs for compaction
- Since paging offers numerous advantages, it is used in most operating systems, from those for large servers through those for mobile devices.
- Paging is implemented through cooperation between the operating system and the computer hardware.

Paging

- **Divide physical memory into fixed-sized blocks called frames**
 - Size is power of 2, between 512 bytes and 16 Mbytes, depending on the computer architecture
- Divide logical memory into blocks of same size called pages
- When a process is to be executed, its pages are loaded into any available memory frames.
- Keep track of all free frames
 - It has great functionality, for example, the logical address space is now totally separate from the physical address space, so a process can have a logical 64-bit address space even though the system has less than 2^{64} bytes of physical memory.

Address Translation Scheme

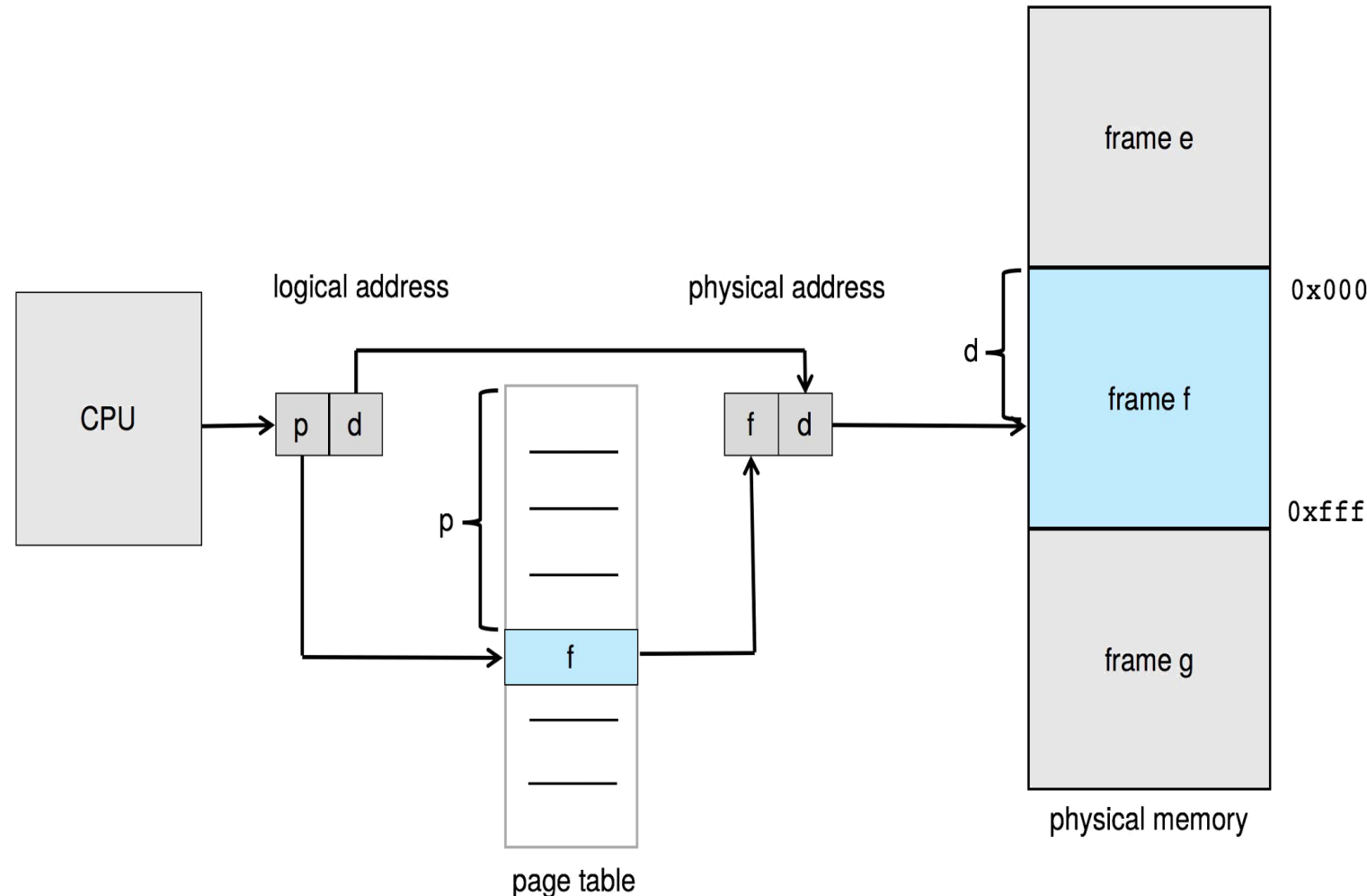
- Address generated by CPU is divided into:
 - **Page number** (p) – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit



- For given logical address space 2^m and page size 2^n
- This information is used by Page Table

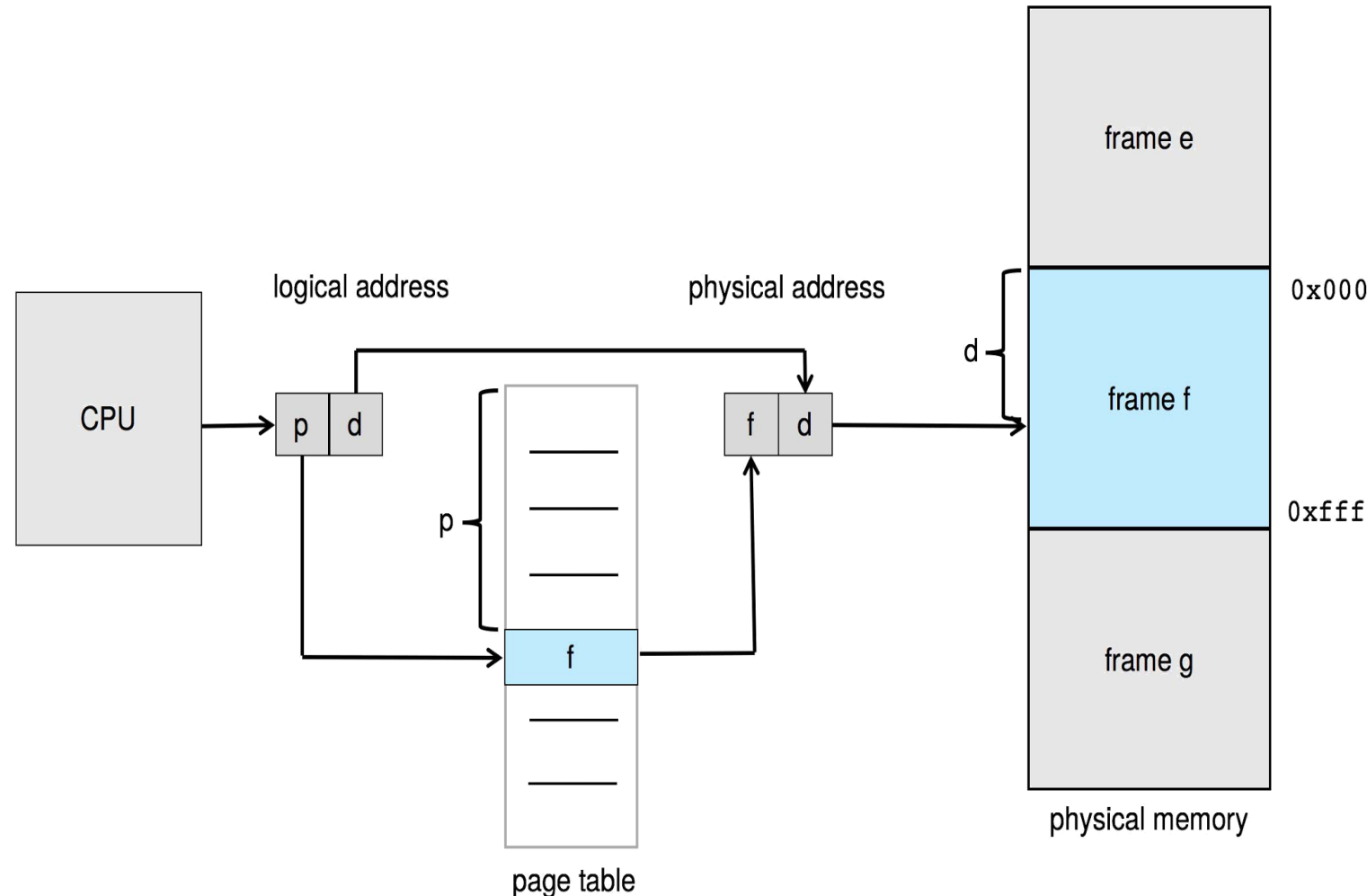
Paging Hardware

- The **page table** is a data structure used by the MMU to translate virtual memory addresses into physical memory addresses.
- The page table contains the base address of each frame in physical memory, and the offset is the location in the frame being references.
- The based address of a frame is combined with the page offset to define the physical memory address.



Paging Hardware

- The steps taken by the MMU to translate a logical address generated by CPU to a physical address:
 - Extract the page number P and use it as an index into the page table.
 - Extract the corresponding frame number F from the page table.
 - Replace the page number P in the logical address with the frame number.



Paging Example

- If the size of the logical address space is 2^m and the page size is 2^n bytes, then the 2^{m-n} is designate the page number.
- Logical address: $n = 2$ and $m = 4$. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Paging Example

- If the size of the logical address space is 2^m and the page size is 2^n bytes, then the 2^{m-n} is designate the page number.
- Logical address: $n = 2$ and $m = 4$. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages).

Logical address 0 is page 0, offset 0. from page table, page 0 is in frame 5. Logical address 0 maps to physical address:

$$20 = (5 \text{ (frame number)} * 4 \text{ (frame size)}) + 0 \text{ (offset)}$$

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Paging Example

- If the size of the logical address space is 2^m and the page size is 2^n bytes, then the 2^{m-n} is designate the page number.
- Logical address: $n = 2$ and $m = 4$. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages).

Logical address 5 is in page 1, offset 1. from page table, page 1 is in frame 6. Logical address 5 maps to physical address:

$$25 = (6 \text{ (frame number)} * 4 \text{ (frame size)}) + 1 \text{ (offset)}$$

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Paging -- Calculating internal fragmentation

- With paging, we have no external fragmentation, but we may have internal fragmentation. For example, considering a case in which:
 - Page size = 2,048 bytes
 - Process size = 72,766 bytes
 - The process will need 35 pages + 1,086 bytes
 - It will be allocated 36 frames with an internal fragmentation of 962 (2,048-1,086)
 - Worst case fragmentation = 1 frame – 1 byte, almost an entire frame
- If the process size is independent of the page size, we expect internal fragmentation to average one-half page per process.
- So, small page size is suggested, but it will result in overhead in page table entry. This overhead is increasing as the page size is decreasing.

Exercise:

- Consider a logical address space of 64 pages of 1,024 words each, mapped onto a physical memory of 32 frames.
 - How many bits are there in the logical address?
 - How many bits are there in the physical address?

Exercise:

- Consider a logical address space of 64 pages of 1,024 words each, mapped onto a physical memory of 32 frames.
 - How many bits are there in the logical address?
 - How many bits are there in the physical address?
- Answer:
 - Logical Address: 16 bits
 - Physical Address: 15 bits

Exercise:

- Assuming a 1.KB page size, what are the page numbers and offsets for the following address references (provided as decimal numbers):
 - 3085
 - 42095
 - 215201
 - 650000
 - 2000001
- Answer:
 - Page = 3, offset = 13,
 - Page = 41, offset = 111,
 - Page = 210, offset = 161
 - Page= 634, offset = 784
 - Page = 1953, offset = 129

Summary

- **Why do we need Memory Management?**

- Program must be brought (from disk) into memory and placed within a process for it to be run

- **Protection (one process does not access the memory of another process).**

- Need to ensure that a process can access only those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** to define the logical address space of a process

- **Logical Memory vs. Physical Memory**

- An address generated by CPU is commonly referred to as a **logical address**
- An address seen by the memory unit is referred to as **physical address**

Summary

- **MMU**

- Hardware device that at run time maps virtual to physical address

- **Contiguous memory allocation**

- each process is contained in a single section of memory that is contiguous to the section containing the next process.

- **Variable Partition – First Fit, Best Fit, Worst Fit**

- First-fit: Allocate the *first* hole that is big enough
- Best-fit: Allocate the *smallest* hole that is big enough
- Worst-fit: Allocate the *largest* hole; must also search entire list

Summary

- Fragmentation
 - As processes are loaded and removed from memory, the free memory space is broken into little pieces.
 - **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous.
 - **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.

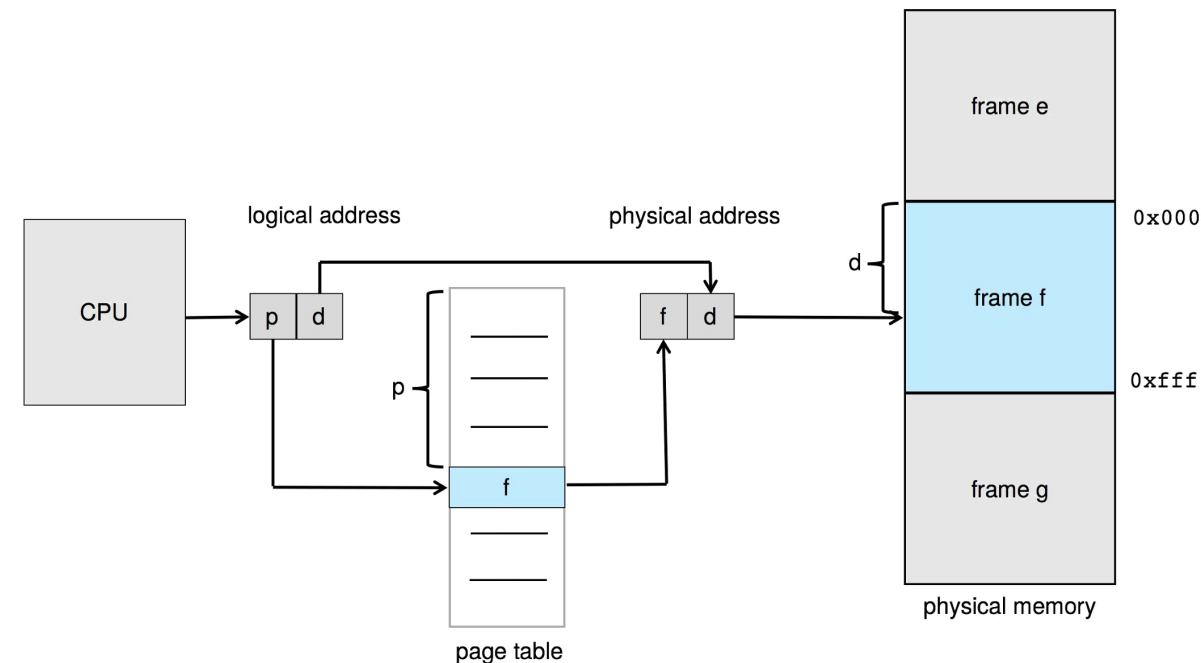
Summary:

- Fragmentation
 - As processes are loaded and removed from memory, the free memory space is broken into little pieces.
 - **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous.
 - **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.
- Paging
 - Permit the logical address space of processes to be **non-contiguous**.
 - Divide physical memory into fixed-sized blocks called **frames**.
 - Divide logical memory into blocks of same size called **pages**.

Summary:

- Page Table

- Address generated by CPU is divided into:
 - Page number (p) – used as an index into a page table which contains base address of each page in physical memory
 - Page offset (d) – combined with base address to define the physical memory address that is sent to the memory unit.

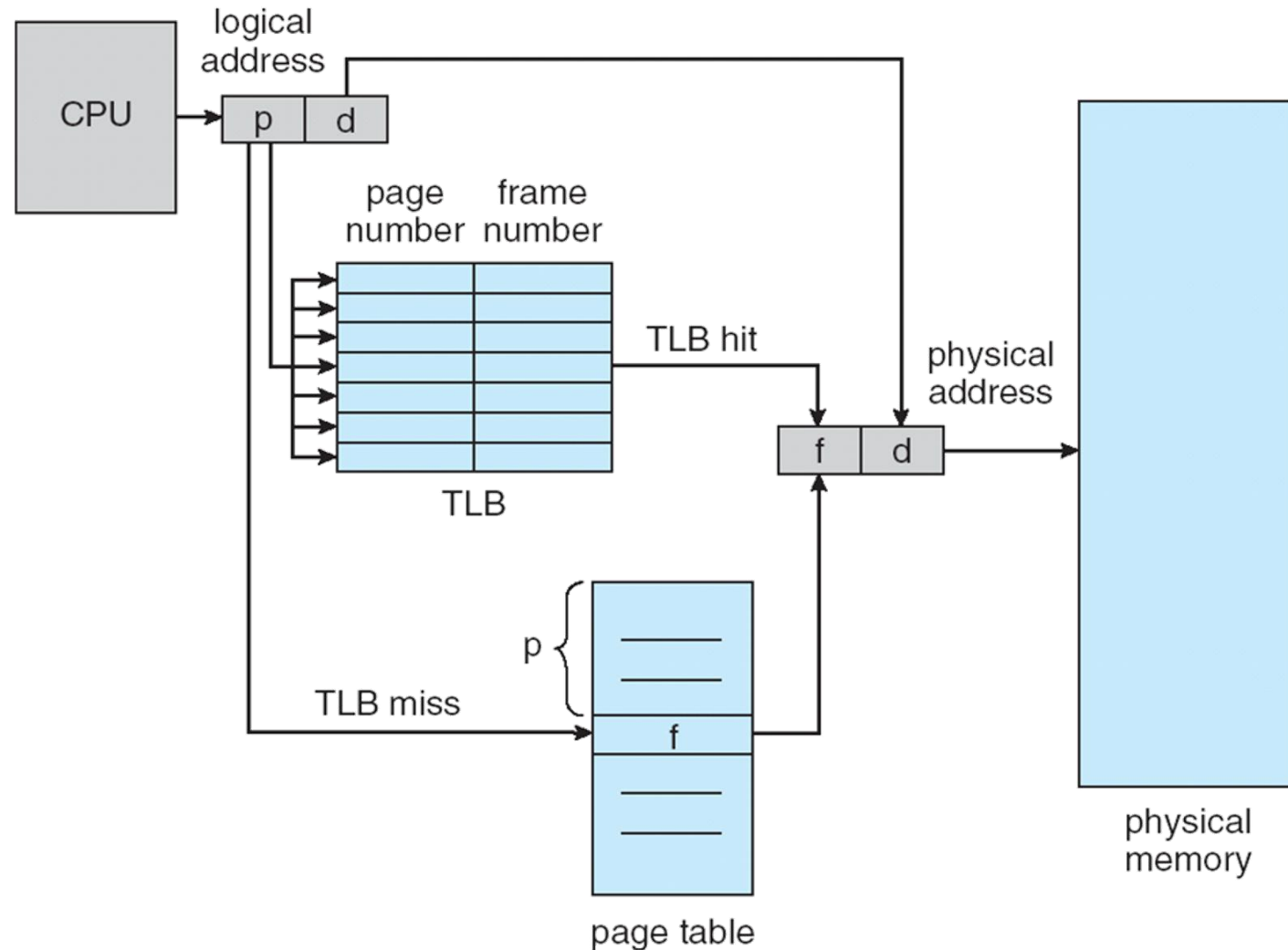


Implementation of Page Table

- Page table is kept in main memory
 - Page-table base register (PTBR) points to the page table (Memory address of page table)
 - Page-table length register (PTLR) indicates the size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data/instruction
- The two-memory access problem can be solved by the use of a special fast-lookup hardware cache called translation look-aside buffers (TLBs).

Paging Hardware With TLB

- The TLB contains only a few of the page-table entry.
- When a logical address is generated by the CPU, the MMU first checks if its page number is present in TLB.
- If the page number is found, its frame number is immediately available and used to access memory.
- These steps adds negligible performance penalty.
- If the page number is not in the TLB, **TLB miss**, address translation is done as before.



Translation Look-Aside Buffer

- Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry – uniquely identifies each process to provide address-space protection for that process
 - Otherwise need to flush at every context switch
- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time
 - Replacement policies must be considered
 - Some entries can be **wired down** for permanent fast access

Effective Access Time

- The percentage of times that the page number of interest is found in the TLB is called the **hit ratio**.
 - An 80% hit ratio means that we find the desired page number in the TLB 80% of the time.
- Suppose that it takes 10 ns to access memory.
 - If we find the desired page in TLB then a mapped-memory access takes 10 ns
 - If we fail to find the page number in the TLB, then we must first access memory for the page table and frame number (10 ns) and then access to desired byte in memory (10 ns), for a total of 20 ns.
- **Effective Access Time (EAT)**
$$\text{EAT} = 0.80 \times 10 + 0.20 \times 20 = 12 \text{ nanoseconds}$$
 - Suffering from a 20% slowdown in average memory access

Effective Access Time

- The percentage of times that the page number of interest is found in the TLB is called the **hit ratio**.
 - An 80% hit ratio means that we find the desired page number in the TLB 80% of the time.
- Suppose that it takes 10 ns to access memory.
- **Effective Access Time (EAT)**

$$\text{EAT} = 0.80 \times 10 + 0.20 \times 20 = 12 \text{ nanoseconds}$$

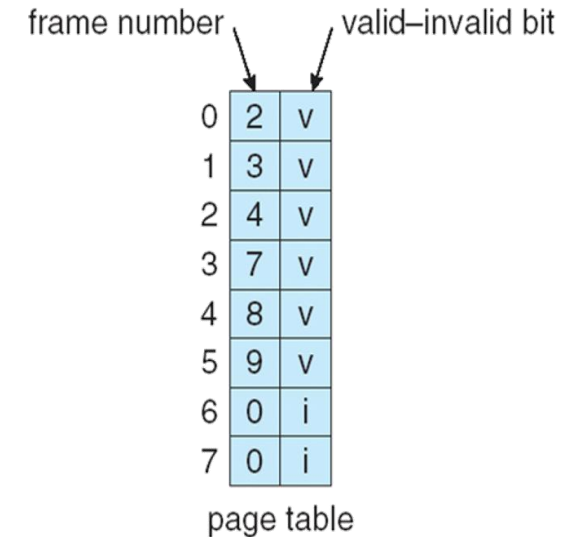
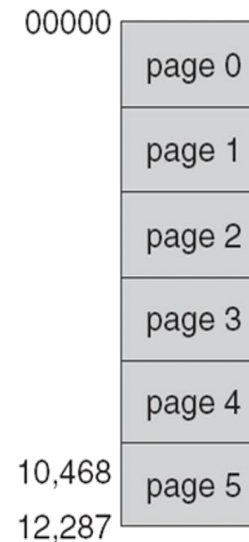
- Suffering from a 20% slowdown in average memory access
- Consider a more realistic hit ratio of 99%,
$$\text{EAT} = 0.99 \times 10 + 0.01 \times 20 = 10.1\text{ns}$$
 - Implying only 1% slowdown in access time.

Memory Protection

- Memory protection in a paged environment is accomplished by **protection bits** associated with each frame.
- These bits are kept in the page table which can define a page to be read-write or read-only.
- Every reference to memory goes through the page table to find the correct frame number.
- At the same time that the physical address is being computed, the protection bits can be checked to verify that no writes are being made to a read-only page.

Memory Protection

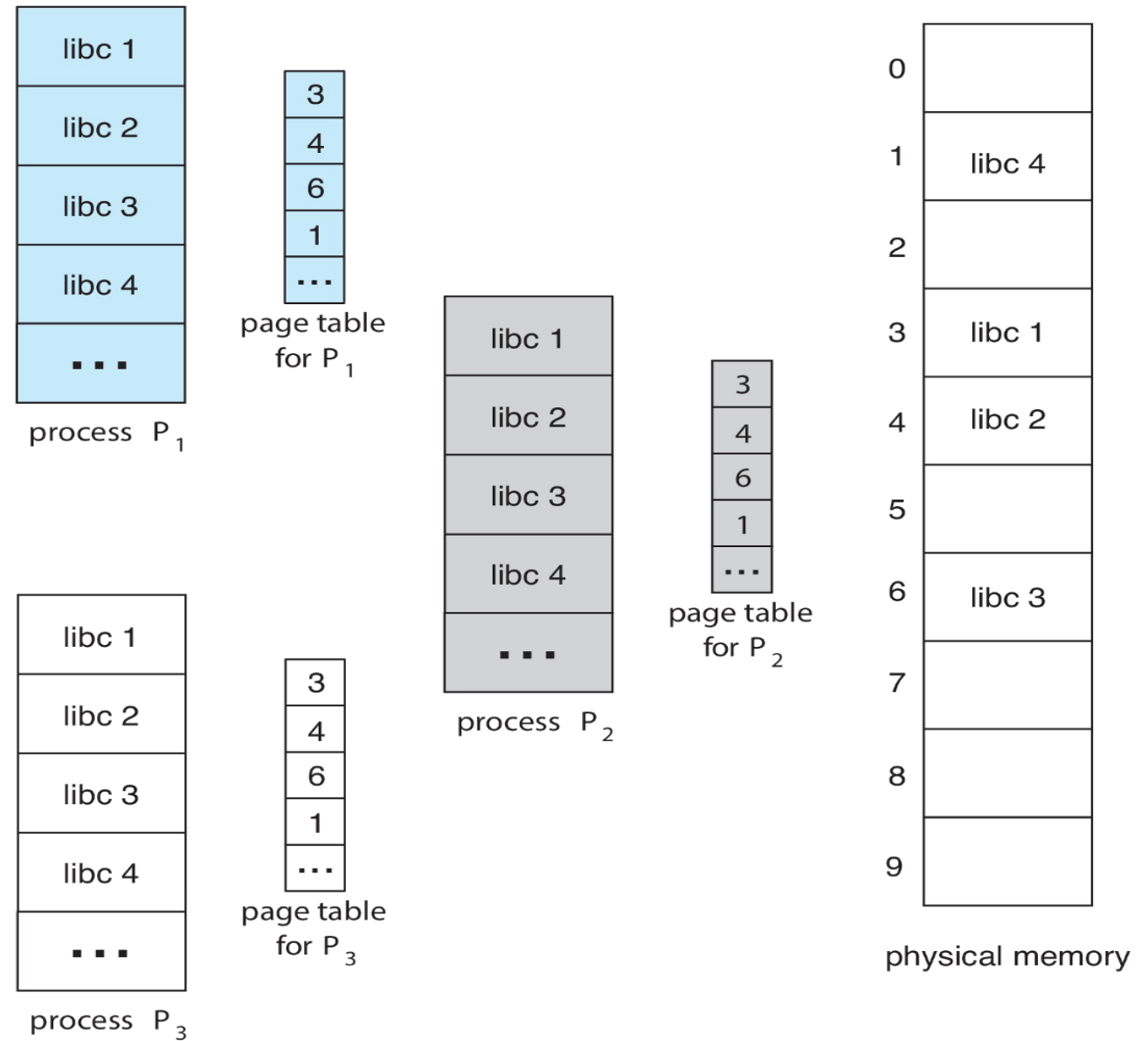
- One additional bit, **Valid-invalid** bit, is attached to each entry in the page table
- When the bit is set to **valid**, the associated page is in the process's logical address and is legal
- When the bit is set to **invalid**, the page is not in the process's logical address space, illegal addresses are trapped.
- The operating system sets this bit for each page to allow or disallow access to the page.



Shared Pages

- An advantage of paging is the possibility of sharing common code which is really important in multiple processes environment.
 - If the code is **reentrant code**, it can be shared among processes.
 - Reentrant code is non-self-modifying code, it never changes during execution, so more processes can share the same code at the same time.
- **Private code and data**
 - Each process keeps a separate copy of the code and data
 - The pages for the private code and data can appear anywhere in the logical address space

Shared Pages Example

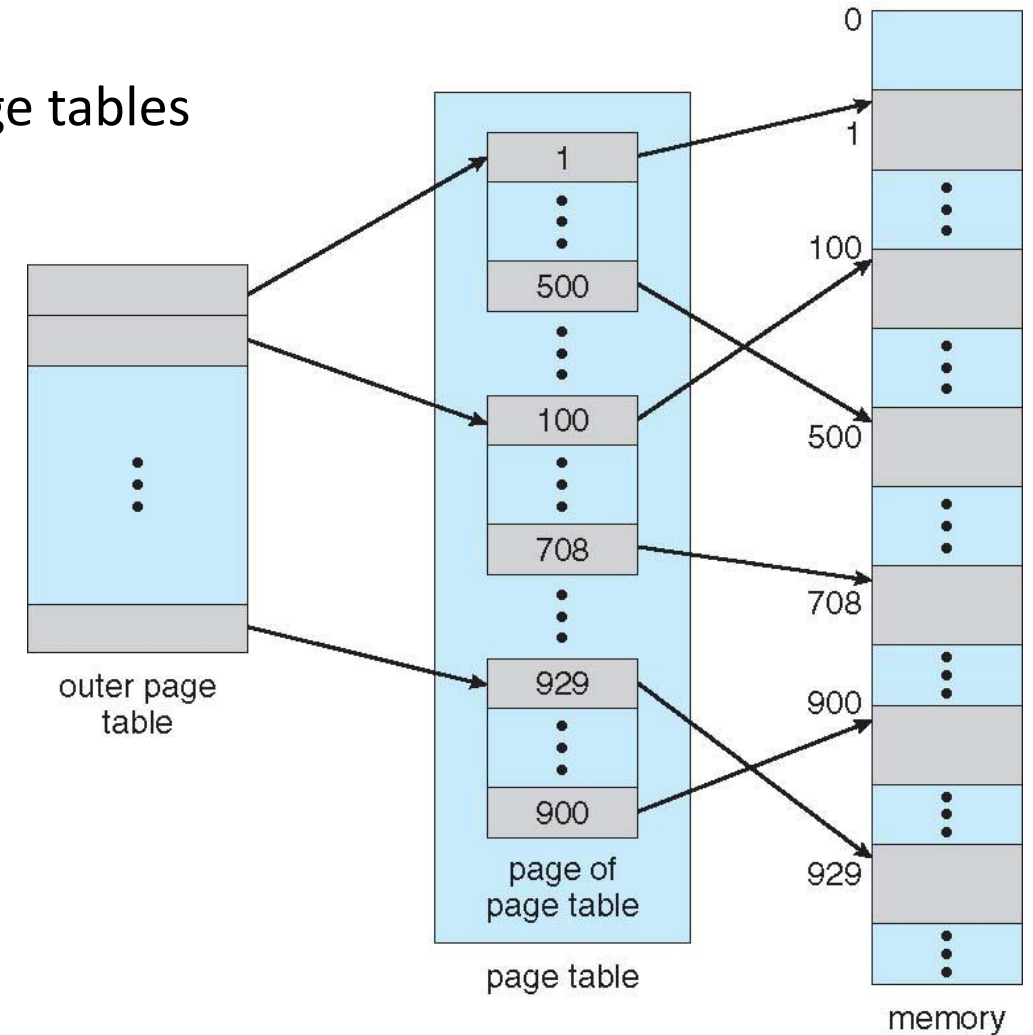


Structure of the Page Table

- Most modern computer systems support a large logical address space. Therefore, the page table itself becomes excessively large.
- Therefore, we would not want to allocate the page table continuously in main memory.
 - Consider a 32-bit logical address space as on modern computers
 - Page size of 4 KB (2^{12})
 - Page table would have 1 million entries ($2^{32} / 2^{12}$)
 - If each entry is 4 bytes → each process 4 MB of physical address space for the page table alone
 - Don't want to allocate that contiguously in main memory
- One simple solution is to divide the page table into smaller units
 - **Hierarchical Paging**
 - **Hashed Page Tables**
 - **Inverted Page Tables**

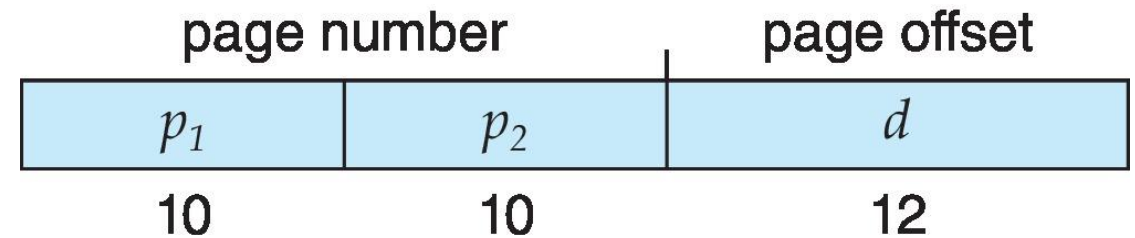
Hierarchical Page Tables

- One solution is to use a two-level paging algorithm, in which the page table itself is also paged.
 - Break up the logical address space into multiple page tables
 - A simple technique is a two-level page table
 - We then page the page table



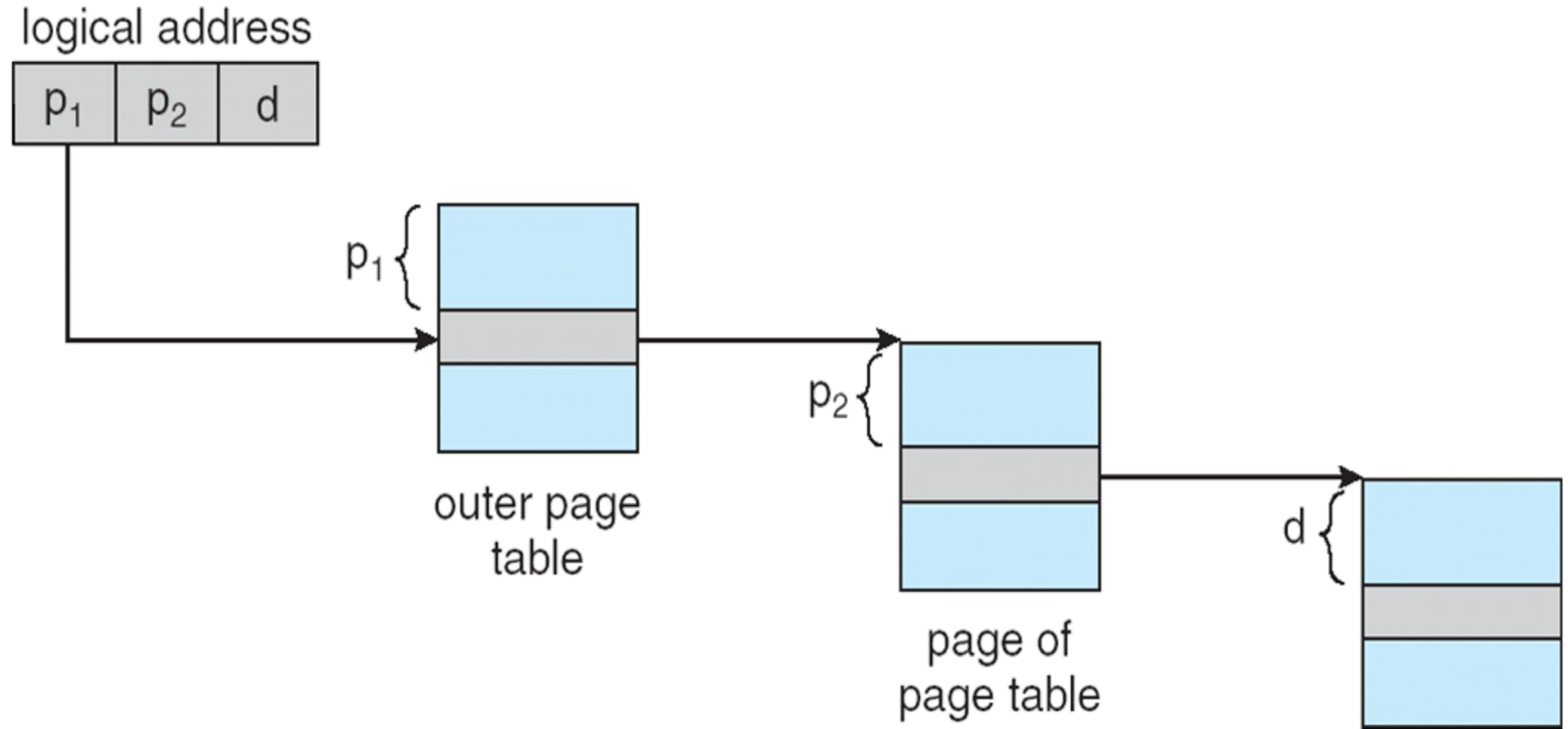
Two-Level Paging Example

- A logical address (on 32-bit machine with 4K page size) is divided into:
 - a page number consisting of 20 bits
 - a page offset consisting of 12 bits
- Since the page table is paged, the page number is further divided into:
 - a 10-bit page number
 - a 10-bit page offset
- Thus, a logical address is as follows:



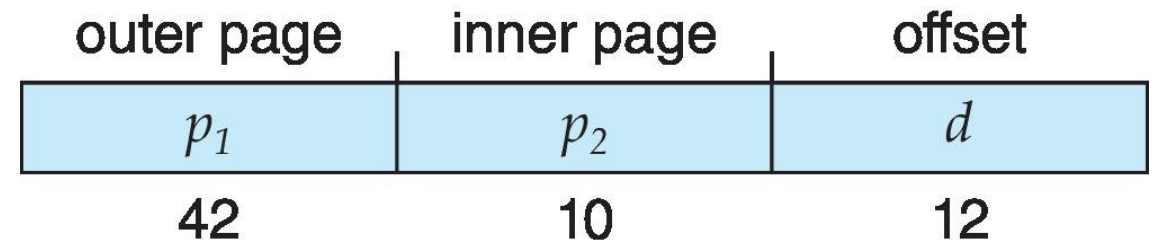
- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table
- Known as **forward-mapped page table**

Address-Translation Scheme



64-bit Logical Address Space

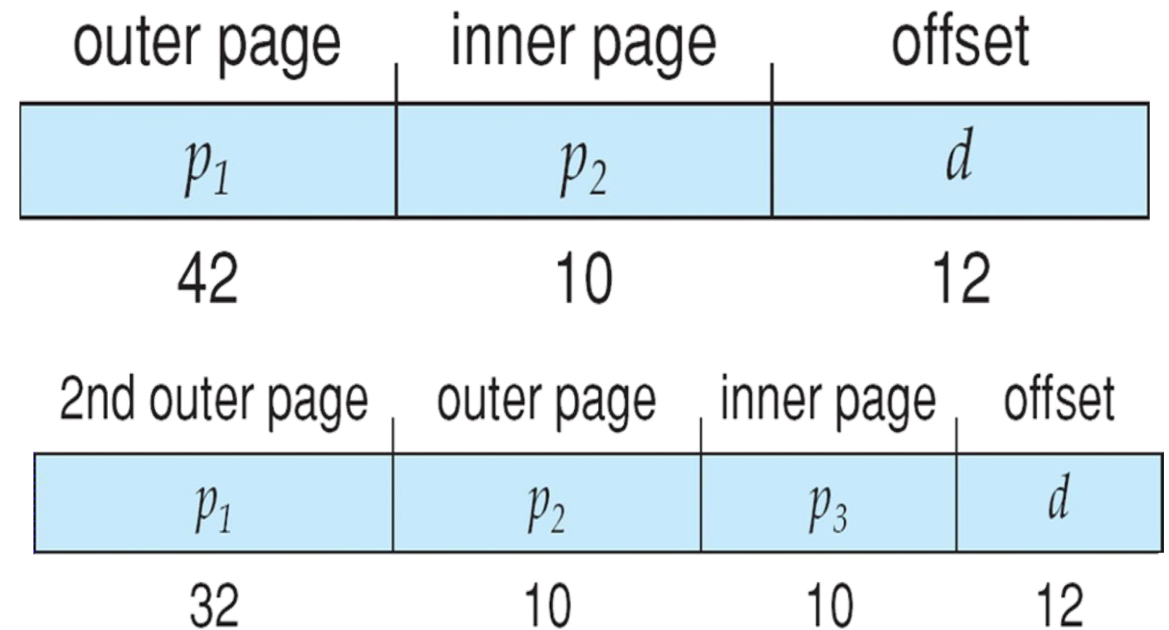
- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
 - Then page table has 2^{52} entries
 - If two level scheme, inner page tables could be 2^{10} 4-byte entries
 - Address would look like



- Outer page table has 2^{42} entries or 2^{44} bytes
- One solution is to add a 2nd outer page table
- But in the following example the 2nd outer page table is still 2^{34} bytes in size
 - And possibly 4 memory access to get to one physical memory location

Three-level Paging Scheme

- The way to avoid such a large table is to divide the outer page table into smaller pieces
- For example, we can page the outer page table, giving us a three-level paging scheme.

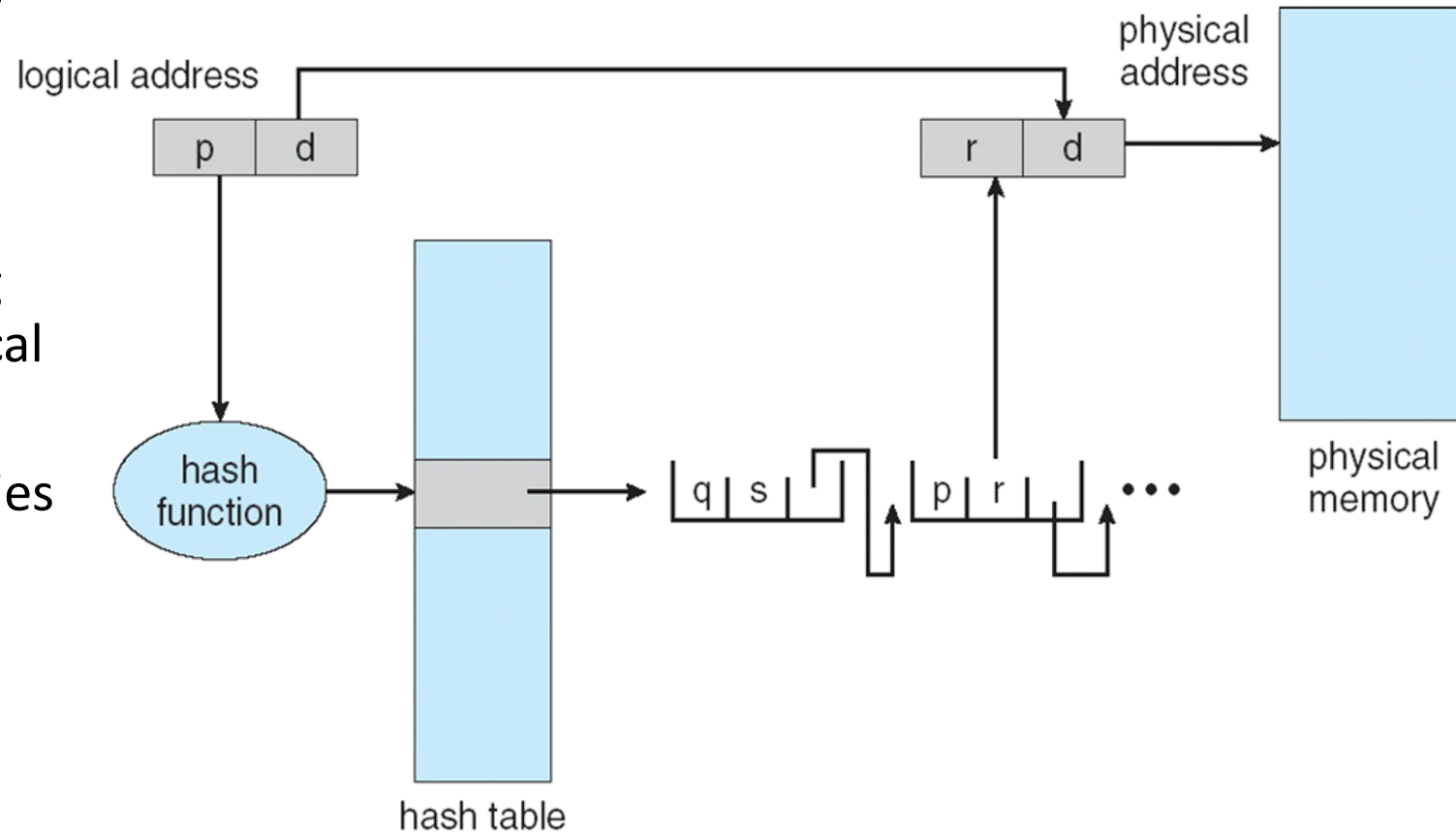


Hashed Page Tables

- One approach for handling address spaces larger than 32 bits is to use a **hashed page table**, with the hash value being the virtual page number.
- Each entry in the hash table contains a linked list of elements that hash to the same location.
- Each elements consists of three fields:
 - 1) The virtual page number
 - 2) the value of the mapped page frame
 - 3) a pointer to the next element in the linked list

Hashed Page Tables

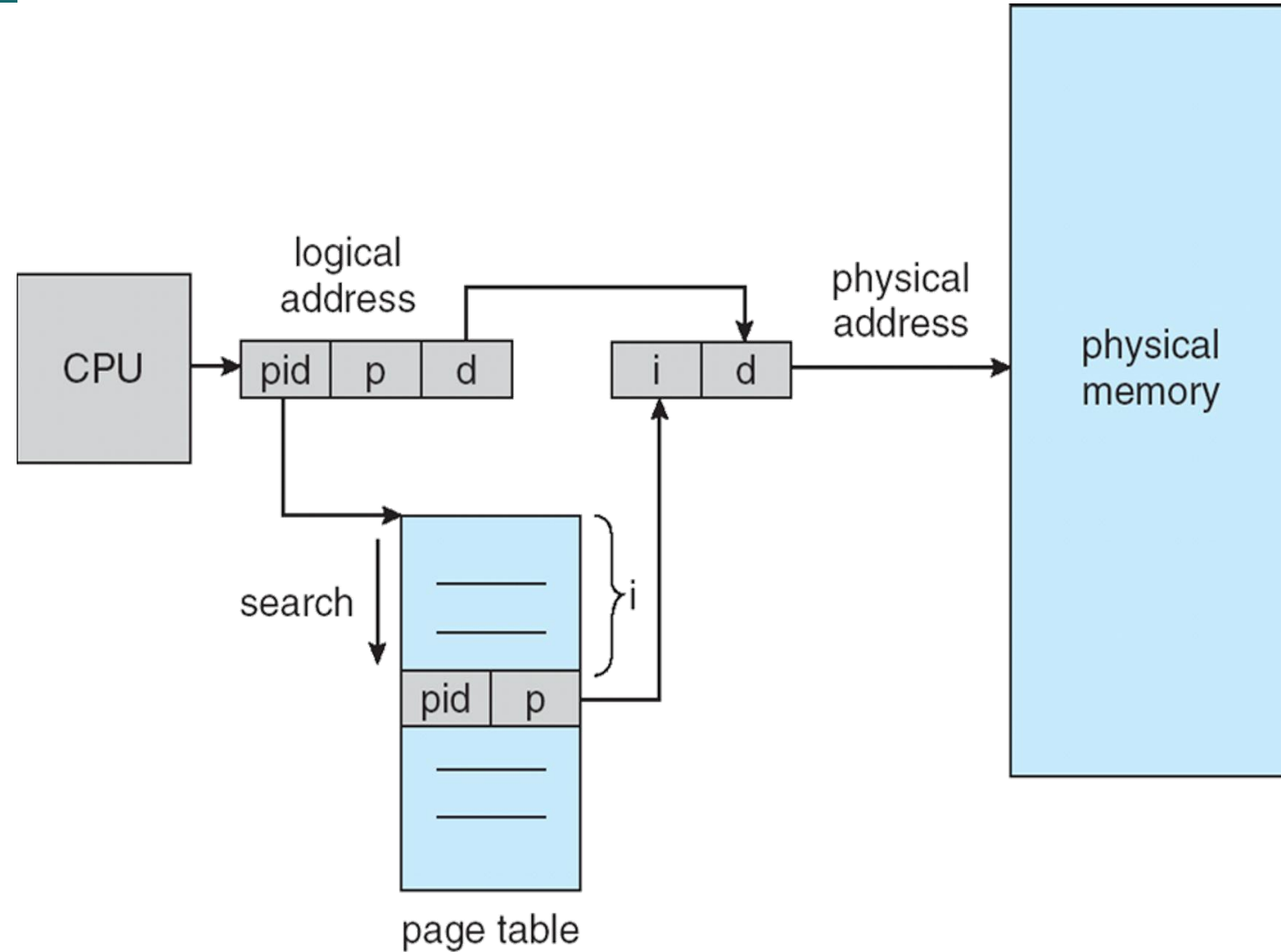
- The virtual page number in the virtual address is hashed into the hash table.
- The virtual page number is compared with field 1 in the first element in the linked list.
- If there is a match, the corresponding page frame is used to form the physical address.
- If there is no match, subsequent entries in the linked list are searched for a matching virtual page number.



Inverted Page Table

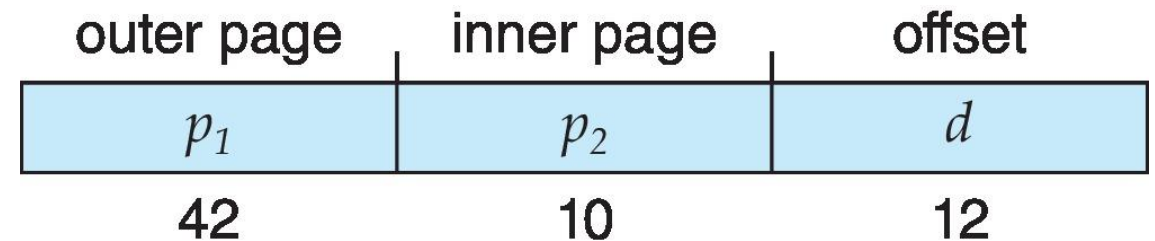
- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
 - TLB can accelerate access
- But how to implement shared memory?
 - One mapping of a virtual address to the shared physical address

Inverted Page Table Architecture



64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
 - Then page table has 2^{52} entries
 - If two level scheme, inner page tables could be 2^{10} 4-byte entries
 - Address would look like



- Outer page table has 2^{42} entries or 2^{44} bytes
- One solution is to add a 2nd outer page table
- But in the following example the 2nd outer page table is still 2^{34} bytes in size
 - And possibly 4 memory access to get to one physical memory location

Exercise

- Consider a computer system with a 21-bit virtual address that has only a 16-bit physical address. It also has a 2KB page size. How many entries are there in each of the following?
- A conventional, single-level page table
- An inverted page table

Exercise

- Consider a computer system with a 21-bit virtual address that has only a 16-bit physical address. It also has a 2KB page size. How many entries are there in each of the following?
- A conventional, single-level page table
- An inverted page table
- Answer:
 - $2^{10} = 1024$ entries (Number of entries = Virtual address space size / Page size)
 - $2^5 = 32$

Exercise

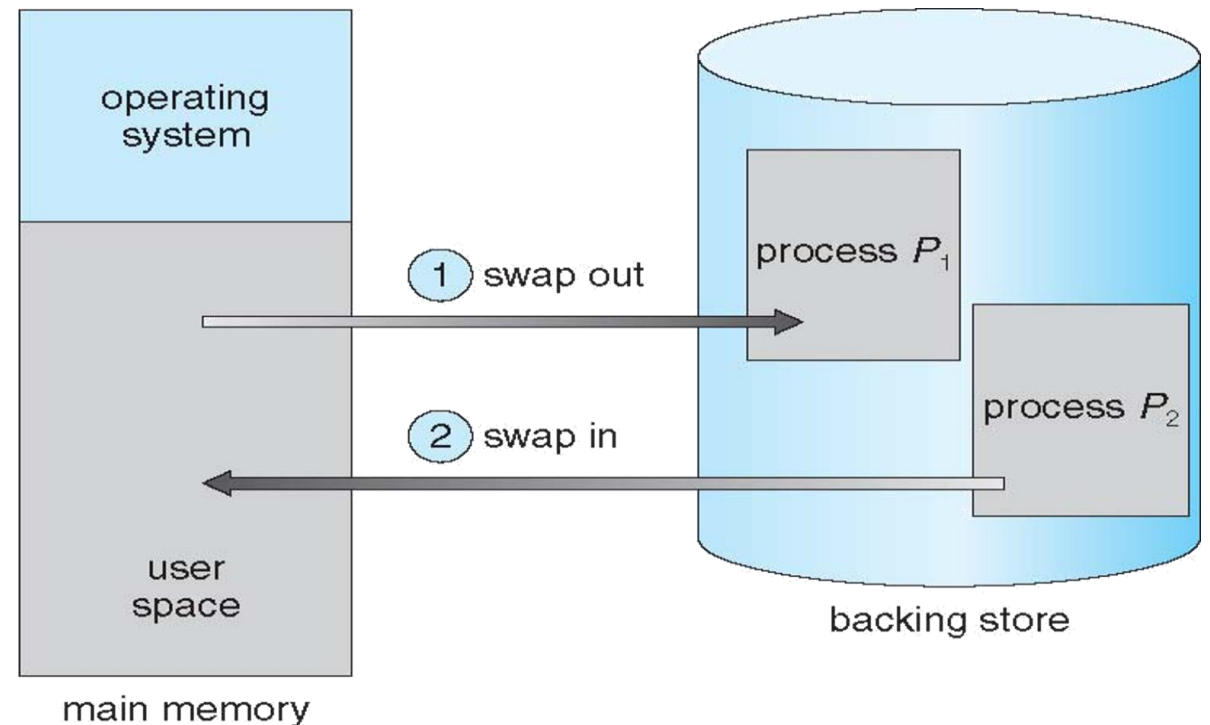
- Consider a computer system with a 32-bit logical address and 4-KB page size. The system supports up to 512 MB of physical memory. How many entries are there in each of the following?
- A conventional, single-level page table
- An inverted page table

Exercise

- Consider a computer system with a 32-bit logical address and 4-KB page size. The system supports up to 512 MB of physical memory. How many entries are there in each of the following?
- A conventional, single-level page table
- An inverted page table
- Answer:
 - 2^{20} entries (Number of entries = Virtual address space size / Page size)
 - 512 MB/4K = 128K entries

Swapping

- Process instructions and data for operating must be in memory to be executed.
- However, a process or a portion of a process can be **swapped** temporarily out of memory to a backing store and then brought back into memory for continued execution.
- Swapping makes it possible for the total physical address space of all processes to exceed the real physical memory of the system.

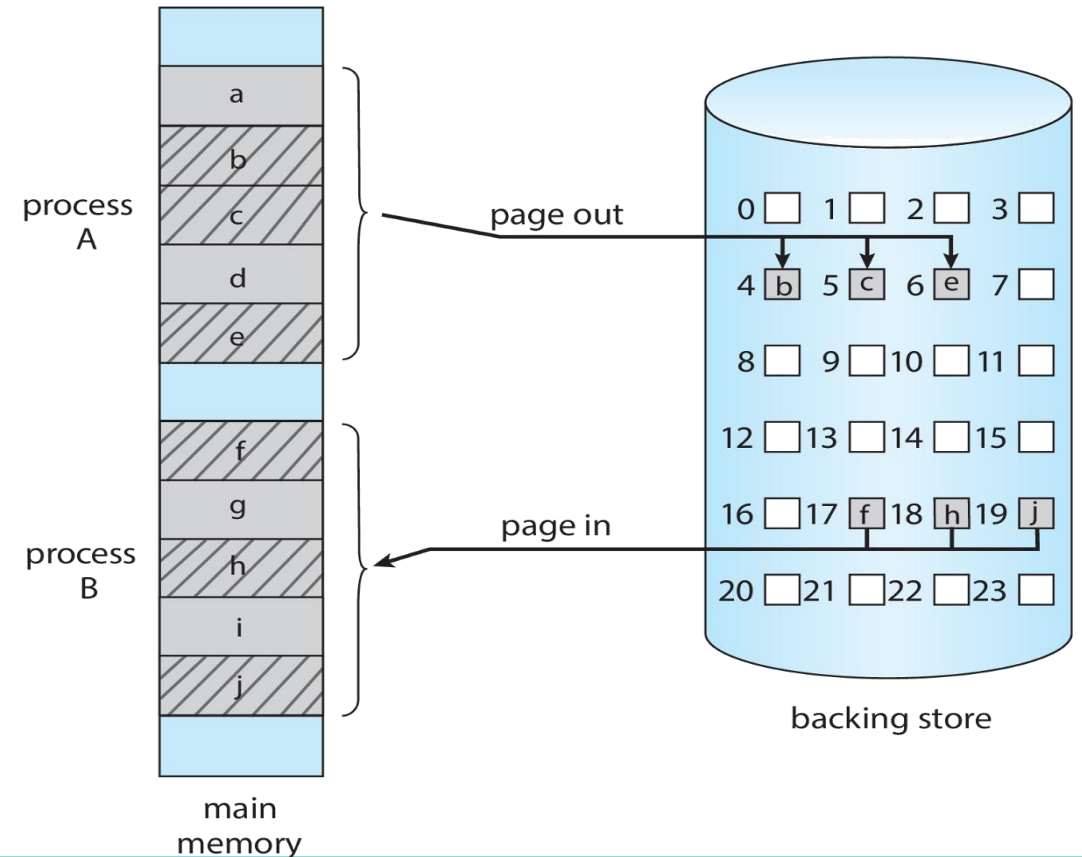


Standard Swapping

- Standard swapping involves moving entire processes between main memory and a backing store.
- The backing store must be large enough to accommodate whatever parts of the process needs to be stored and retrieved.
- When a process is swapped to the backing store, the data structures associated with the process must be written to the backing store.
- The advantages is that it allows physical memory to be oversubscribed, so the system can accommodate more processes than there is actual physical memory.

Swapping with Paging

- A problem with standard swapping is that the amount of time required to move entire process between memory and the backing store is expensive.
- Therefore, instead of the entire process, pages of a process can be swapped.
- A **page out** operation moves a page from a memory to the backing stored.
- The reverse process is known as **page in**



Swapping on mobile systems

- Typically, mobile systems do not support swapping.
 - Mobile devices generally use flash memory rather than more spacious hard disks with limited number of writes that it can tolerate before it becomes reliable.
- Instead of swapping, when free memory falls below threshold:
 - Apple iOS asks application to voluntarily relinquish allocated memory.
 - The application that fails to free up sufficient memory may be terminated by the operating system.
 - Android may terminate a process if insufficient free memory is available. However, before terminating, android writes its application state to flash memory so it can be quickly restarted.

Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
 - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk

Swapping (Cont.)

- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
 - Plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
 - Swapping normally disabled
 - Started if more than threshold amount of memory allocated
 - Disabled again once memory demand reduced below threshold

Summary

- **Why do we need Memory Management?**

- Program must be brought (from disk) into memory and placed within a process for it to be run

- **Protection (one process does not access the memory of another process).**

- Need to ensure that a process can access only those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** to define the logical address space of a process

- **Logical Memory vs. Physical Memory**

- An address generated by CPU is commonly referred to as a **logical address**
- An address seen by the memory unit is referred to as **physical address**

Summary

- **MMU**

- Hardware device that at run time maps virtual to physical address

- **Contiguous memory allocation**

- each process is contained in a single section of memory that is contiguous to the section containing the next process.

- **Variable Partition – First Fit, Best Fit, Worst Fit**

- First-fit: Allocate the *first* hole that is big enough
- Best-fit: Allocate the *smallest* hole that is big enough
- Worst-fit: Allocate the *largest* hole; must also search entire list

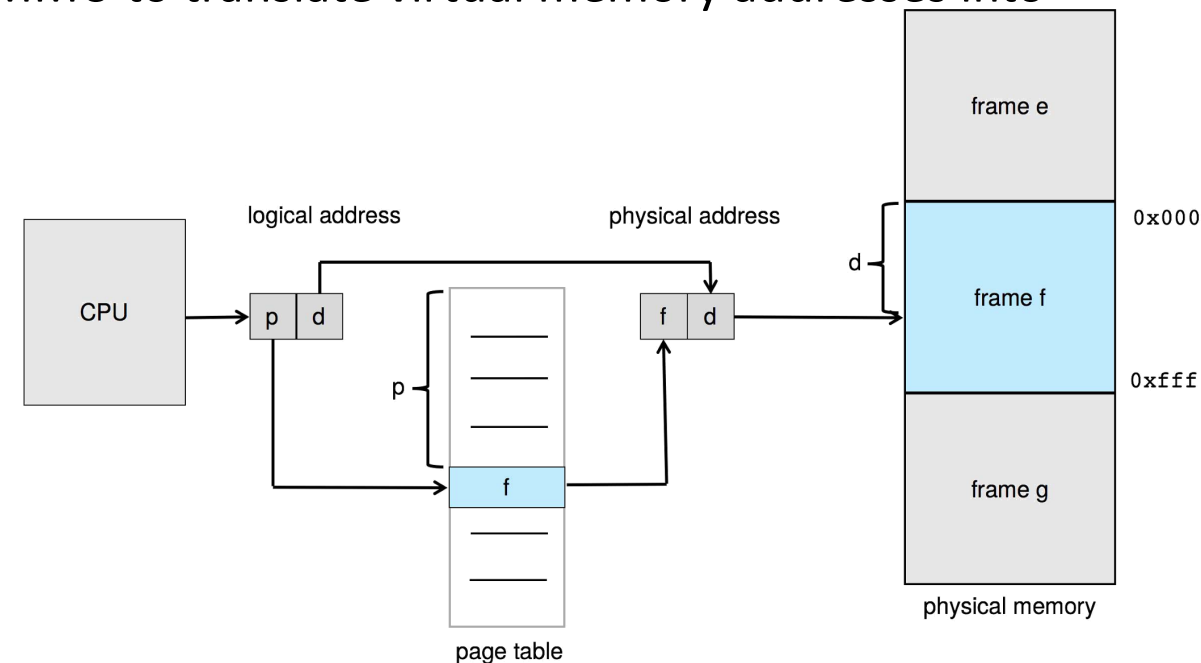
Summary:

- Fragmentation
 - As processes are loaded and removed from memory, the free memory space is broken into little pieces.
 - **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous.
 - **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.
- Paging
 - Permit the logical address space of processes to be **non-contiguous**.
 - Divide physical memory into fixed-sized blocks called **frames**.
 - Divide logical memory into blocks of same size called **pages**.

Summary:

- Page Table

- Address generated by CPU is divided into:
 - **Page number** (p) – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit.
 - The **page table** is a data structure used by the MMU to translate virtual memory addresses into physical memory addresses.



Summary:

- Implementation of Page Table
 - **Page-table base register (PTBR)** points to the page table (Memory address of page table)
 - **Page-table length register (PTLR)** indicates the size of the page table.
- Translation look-aside buffer (TLB)
 - The two-memory access problem can be solved by the use of a special fast-lookup hardware cache called **translation look-aside buffers (TLBs)**.
- Effective Access time
 - The percentage of times that the page number of interest is found in the TLB is called the **hit ratio**.

Summary:

- Implementation of Page Table

- Hierarchical Paging

- Break up the logical address space into multiple page tables
 - A simple technique is a two-level page table

- Hashed Page Tables

- Translating logical memory addresses into physical memory locations by using a hash function to map logical page numbers directly to physical frame numbers.

- Inverted Page Tables

- Mapping physical memory locations to virtual addresses by storing entries for each physical page frame.

page number		page offset
p_1	p_2	d
10	10	12

Summary:

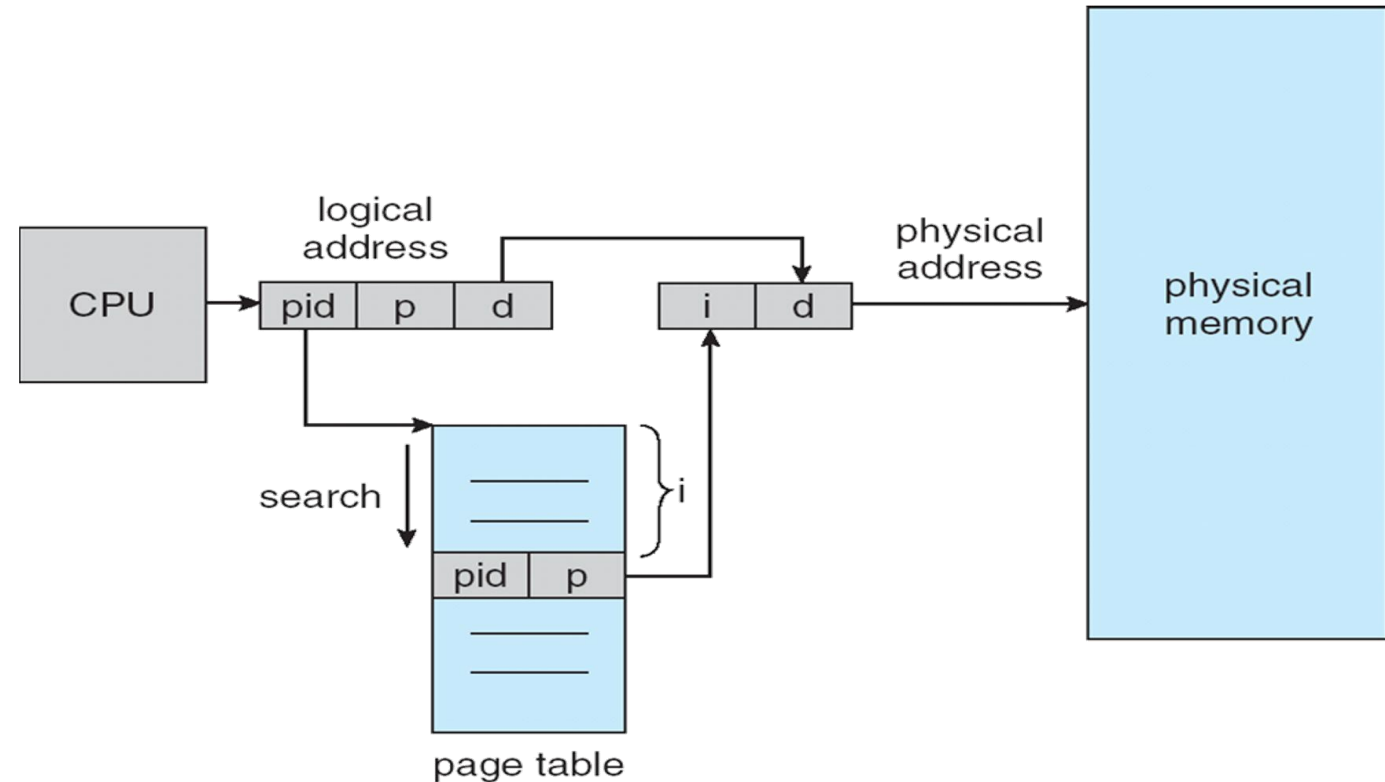
- Implementation of Page Table

- Hierarchical Paging

- Hashed Page Tables

- Inverted Page Tables

- Mapping physical memory locations to virtual addresses by storing entries for each physical page frame.



Summary:

- Swapping
 - Involving temporarily moving data between RAM and secondary storage (e.g., hard disk) to free up space in RAM.
 - A **page out** operation moves a page from a memory to the backing stored.
 - The reverse process is known as **page in**

Example: The Intel 32 and 64-bit Architectures

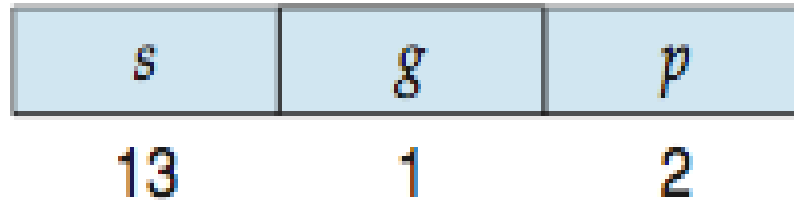
- Currently, the most popular PC operating systems are run on Intel chip such as Linux, windows, macOS.
- Intel architecture has not dominance to mobile systems where the ARM architecture currently is successful.
- Pentium CPUs are 32-bit and called IA-32 architecture
- Current Intel CPUs are 64-bit and called IA-64 architecture
- Many variations in the chips, cover the main ideas here

Example: The Intel IA-32 Architecture

- Supports both segmentation and segmentation with paging
 - The IA-32 architecture allows a segment to be as large as 4GB
 - The maximum number of segments per process is 16K
 - The logical address space of a process is divided into two partitions:
 - First partition consists of up to 8 K segments are private to process, information about the first partition is kept in the [local descriptor table \(LDT\)](#).
 - Second partition of up to 8K segments shared among all processes, information about the second partition is kept in [global descriptor table \(GDT\)](#).

Example: The Intel IA-32 Architecture

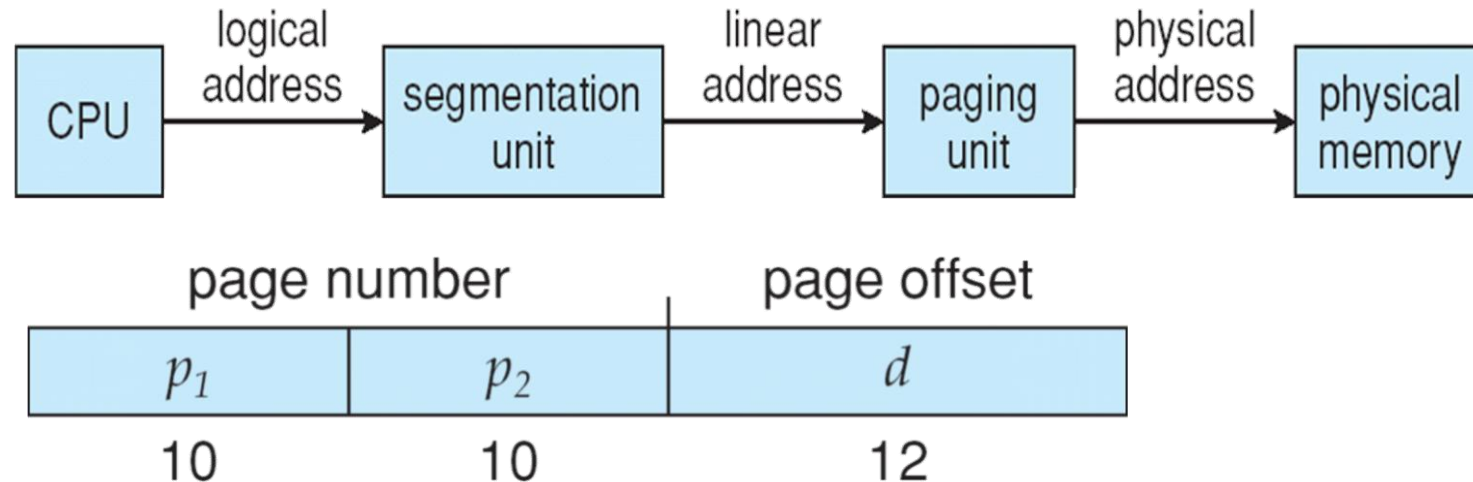
- CPU generates logical address
 - Selector given to segmentation unit
 - Which produces linear addresses



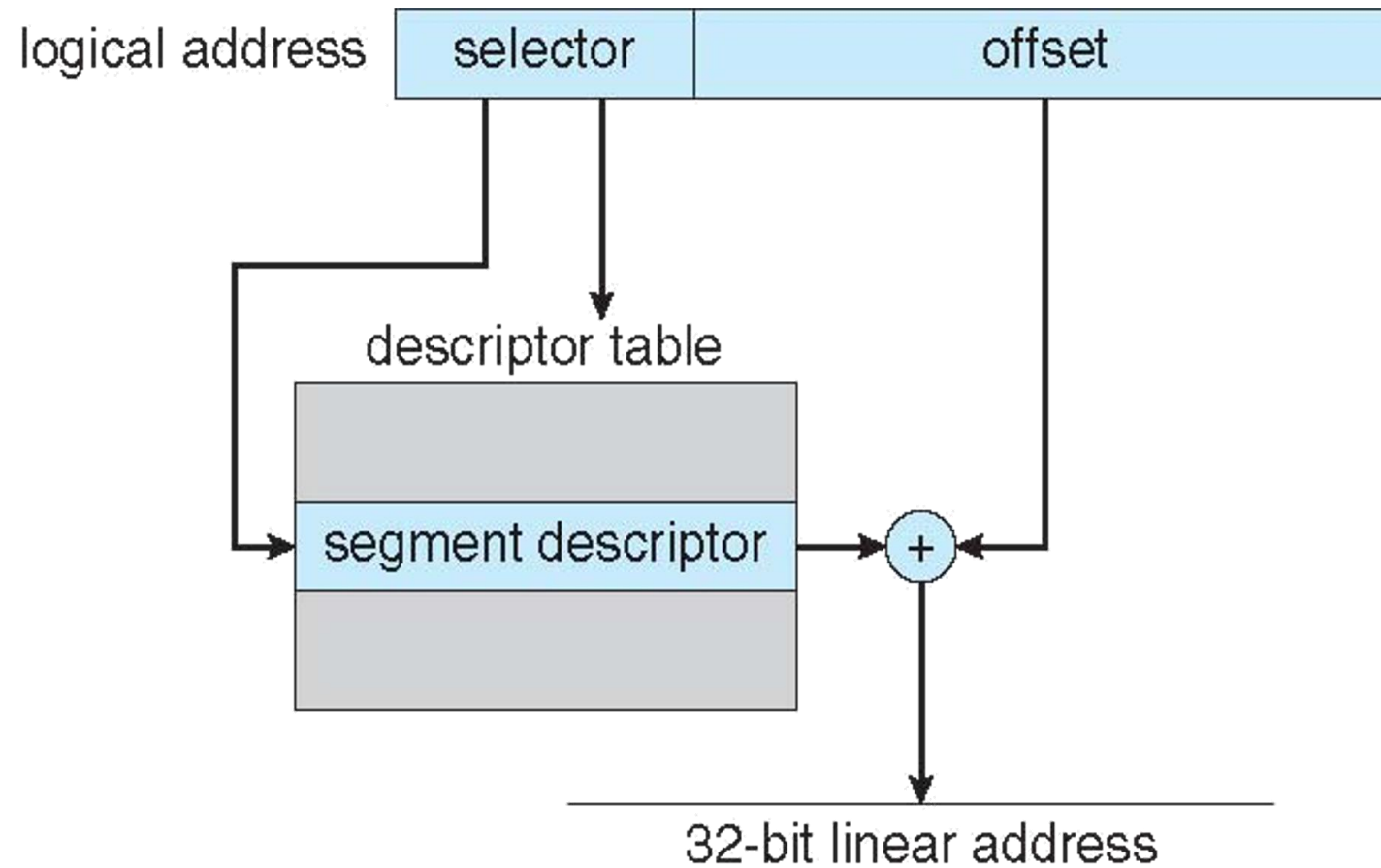
- Linear address given to paging unit
 - Which generates physical address in main memory
 - Paging units form equivalent of MMU
 - **Pages sizes can be 4 KB or 4 MB**

Logical to Physical Address Translation in IA-32

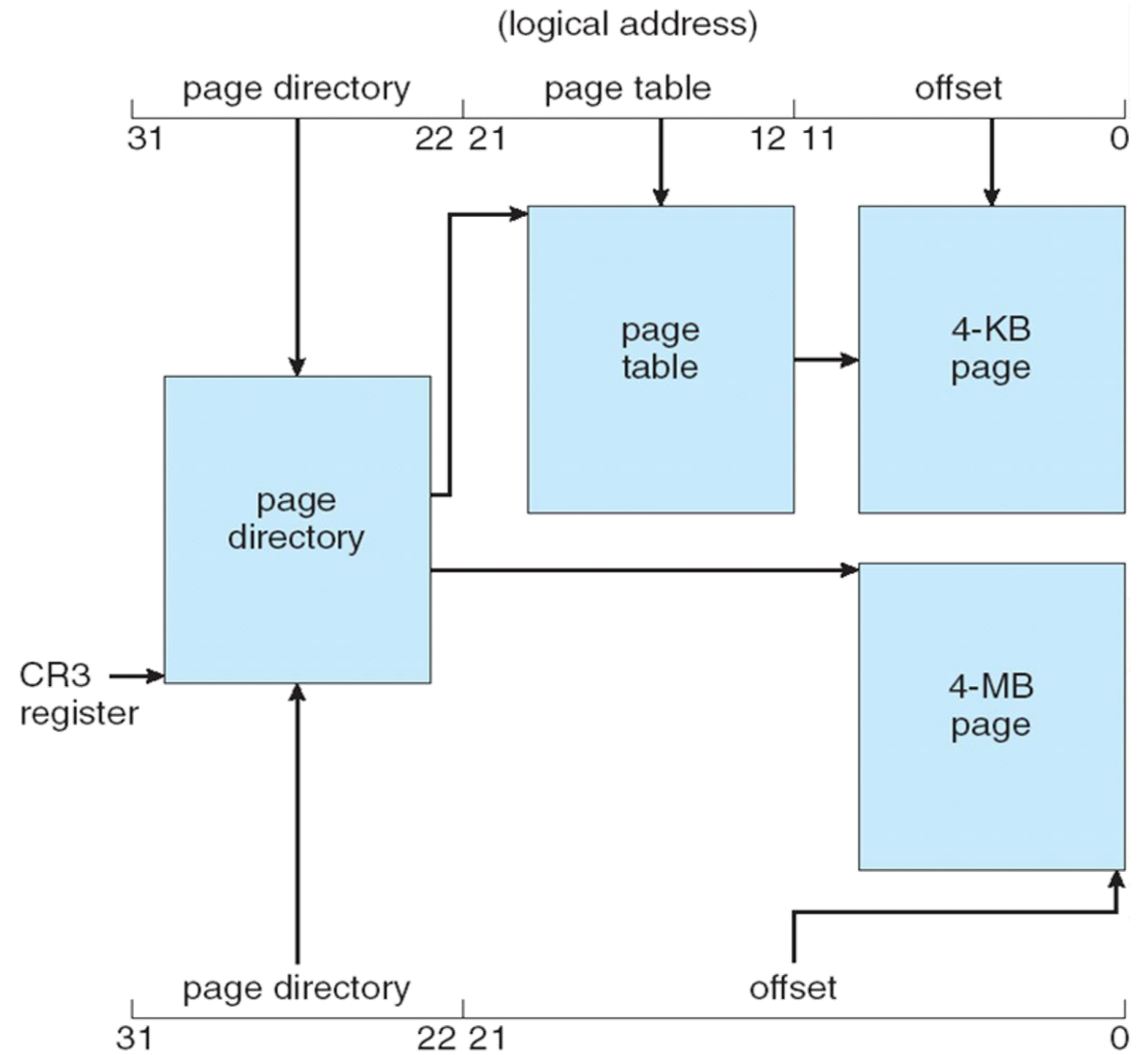
- The logical address pass through the segmentation unit, creating a linear address which will pass through the paging unit for being mapped on physical memory.



Intel IA-32 Segmentation

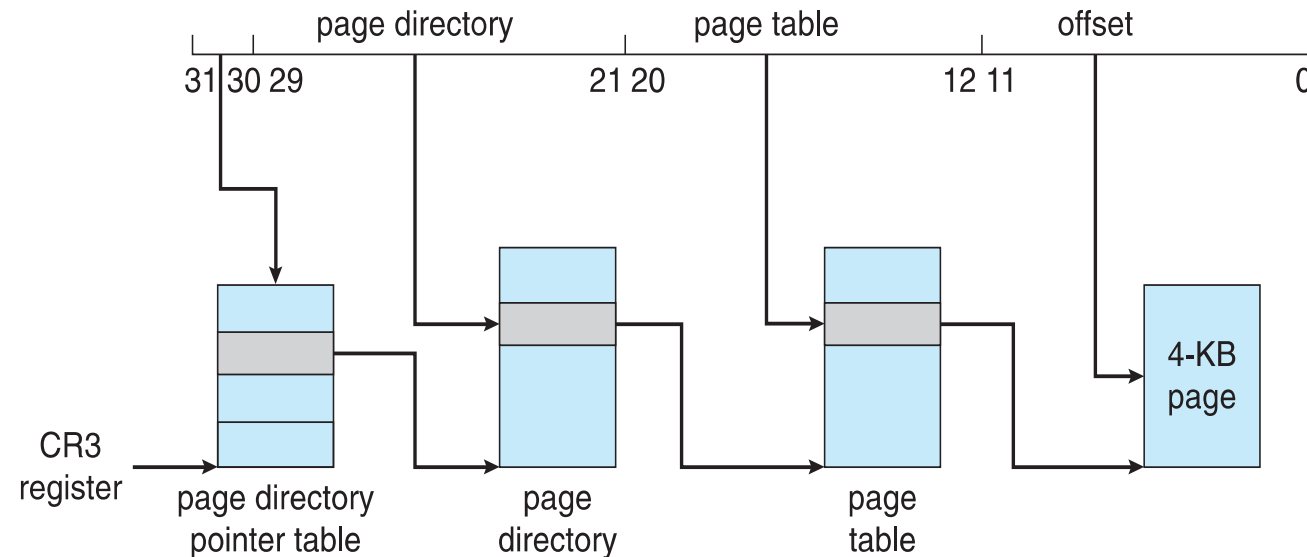


Intel IA-32 Paging Architecture



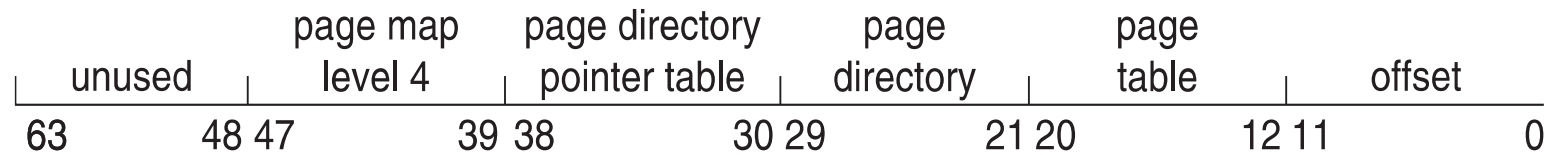
Intel IA-32 Page Address Extensions

- 32-bit address limits led Intel to create **page address extension (PAE)**, allowing 32-bit apps access to more than 4GB of memory space
 - Paging went to a 3-level scheme
 - Top two bits refer to a **page directory pointer table**
 - Page-directory and page-table entries moved to 64-bits in size
 - Net effect is increasing address space to 36 bits – 64GB of physical memory



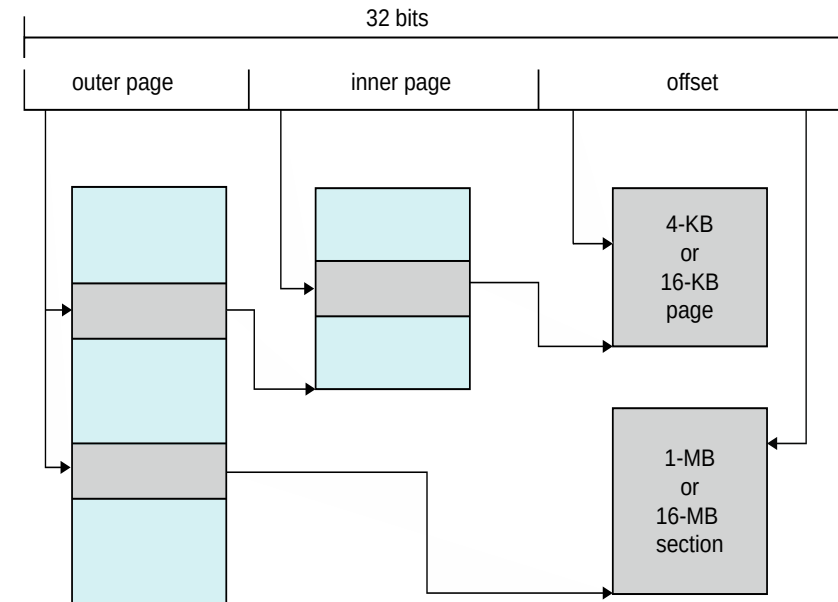
Intel x86-64

- Current generation Intel x86 architecture
- Support for a 64-bit address space yields an astonishing 264 bytes of addressable memory which is huge.
- In practice only implement 48 bit addressing
 - Page sizes of 4 KB, 2 MB, 1 GB
 - Four levels of paging hierarchy
- Can also use PAE so virtual addresses are 48 bits and physical addresses are 52 bits



Example: ARM Architecture

- Dominant mobile platform chip (Apple iOS and Google Android devices for example)
- Modern, energy efficient, 32-bit CPU
- 4 KB and 16 KB pages
- 1 MB and 16 MB pages (termed **sections**)
- Up to four level of paging may be used
- One-level paging for sections, two-level for smaller pages
- Two levels of TLBs
 - Outer level has two micro TLBs (one data, one instruction)
 - Inner is single main TLB
 - First inner is checked, on miss others are checked, and on miss page table walk performed by CPU



Thanks



**Politecnico
di Torino**

Department of Control and
Computer Engineering



Questions?

sarah.azimi@polito.it